

CS360 Introduction to Database

Index Structures

Min-Soo Kim

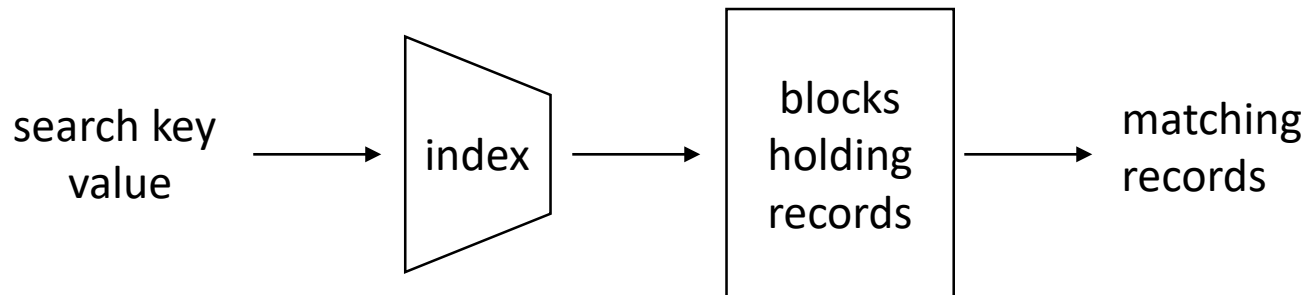
School of Computing, KAIST

Outline

- Dense and sparse index
- Secondary index
- B-tree index
- Hash table and index
- Multidimensional index
- Bitmap index

Index

- Speed up queries that specify values for one or more attributes
- Any data structure can be an index
 - take the value of one or more fields
 - find the records with that value quickly
- **Key** (search key)
 - the field(s) on whose values the index is constructed
 - c.f., sort key, primary key, ...



Terms

- Data file

- store a relation
- may have one or more index files

- Index file

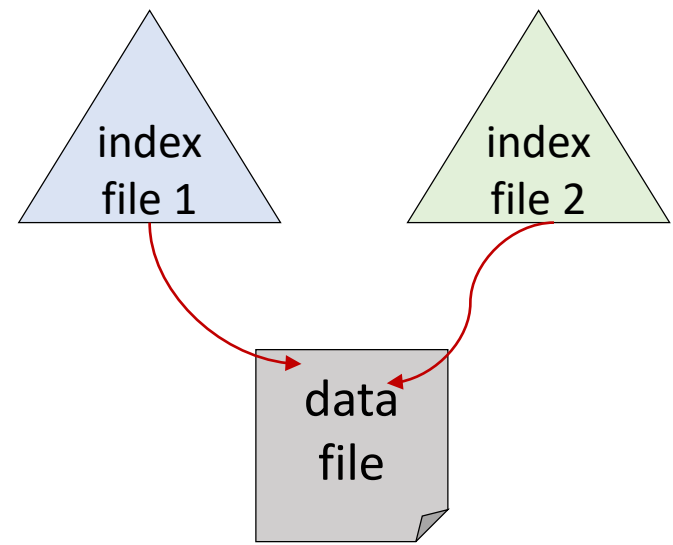
- store an index
- contain the values of the search key with pointers to the records in the corresponding data file

- Dense vs. sparse index

- dense: every record of the data file have index entries in the index
- sparse: only some of the data records have index entries in the index

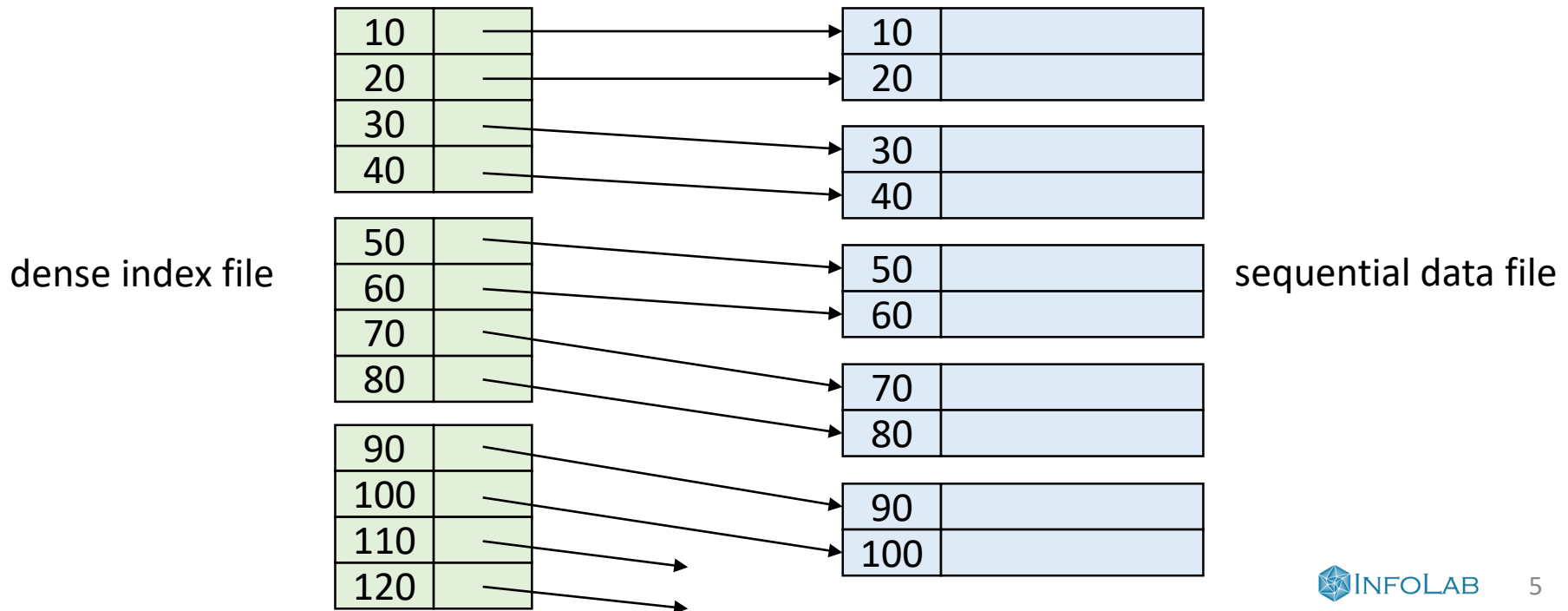
- Primary vs. secondary index

- primary: determine the location of the records
- secondary: NOT determine the location of the records



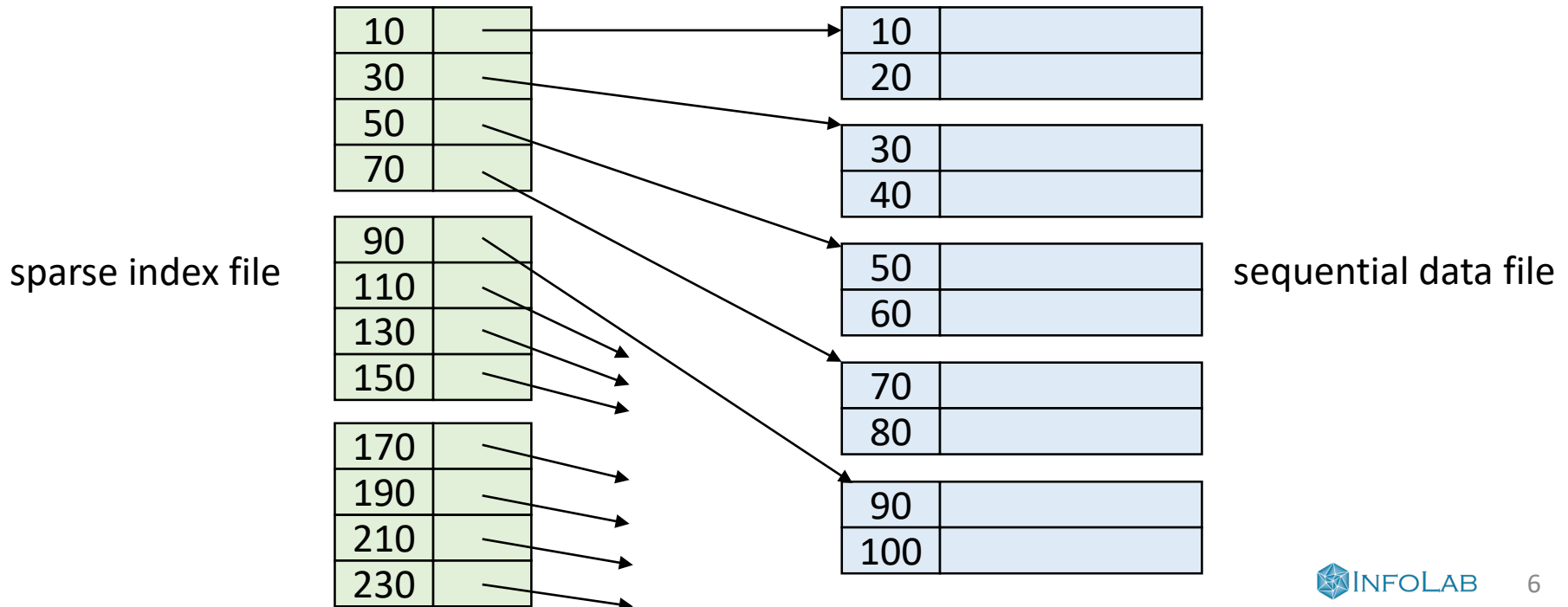
Dense index

- Sequential file
 - created by sorting the tuples of a relation by their primary key
- Dense index: a sequence of index blocks
 - keys are in the **same sorted order** as in the file itself
 - keys and pointers take much less space than complete records



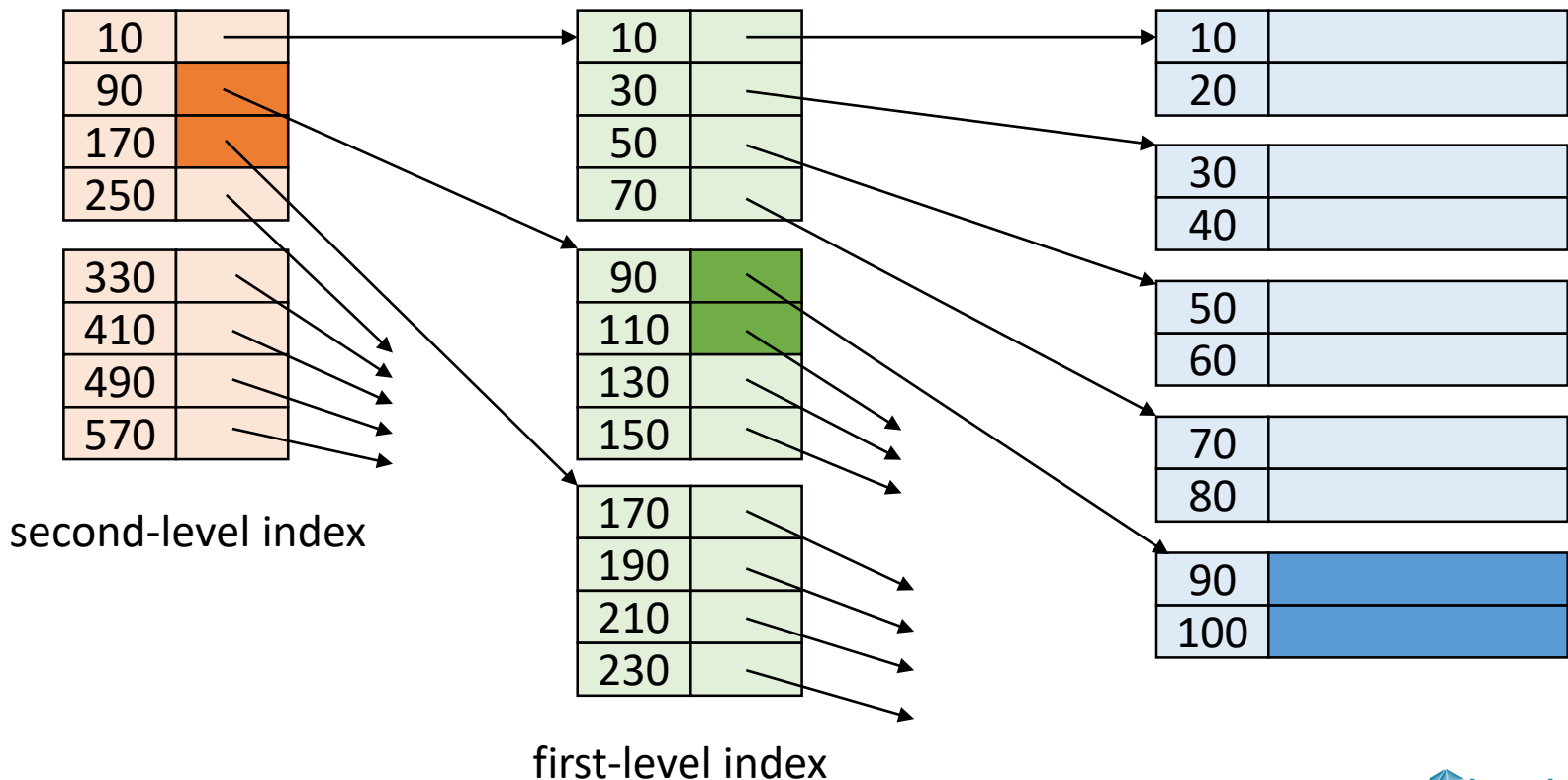
Sparse index

- Typically only one key-pointer pair per block
 - less space than a dense index
 - somewhat more time to find a record given its key
 - can only use if the data file is sorted by the search key
 - c.f., dense index can be used for any search key



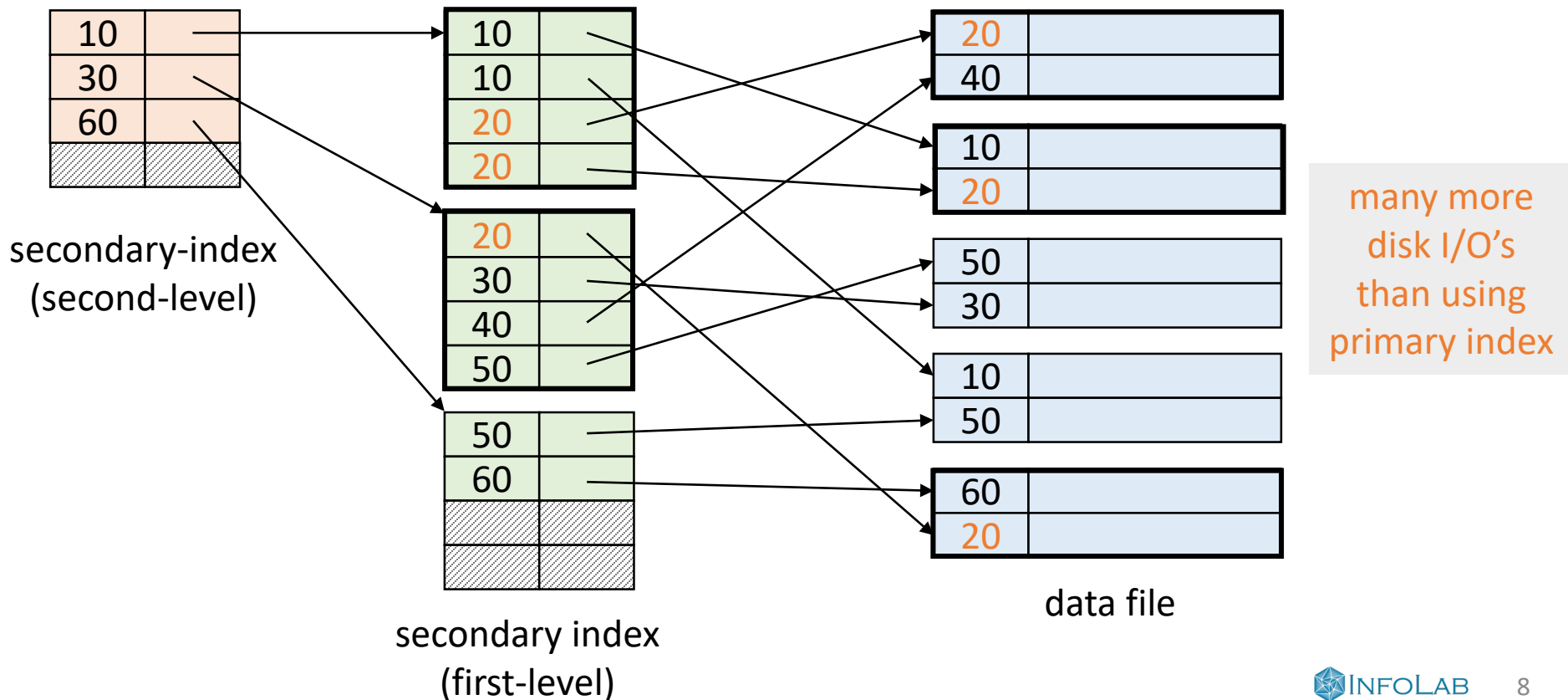
Multiple levels of index

- Using binary search to find the index entry needs many disk I/O's
- Second and higher levels must be sparse
 - otherwise, it has the same size with the first-level index



Secondary index

- NOT determine the placement of records in the data file
 - pointers in one index block can go to many different data blocks
- The first level of secondary index is always dense



Applications of Secondary indexes

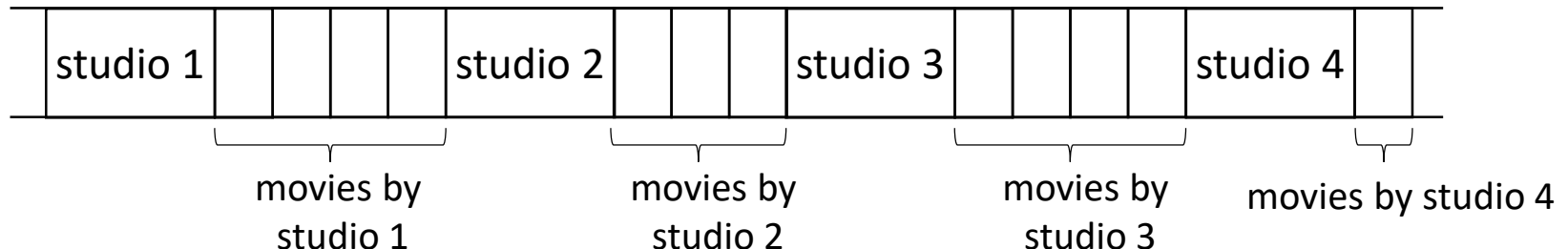
- Clustered file

- we can find the movies for the studio with almost as few disk I/O's as possible by creating an index for Studio.presidentID

M Movie (title, year, length, genre, studioName)
:
1 Studio (name, address, **presidentID**)

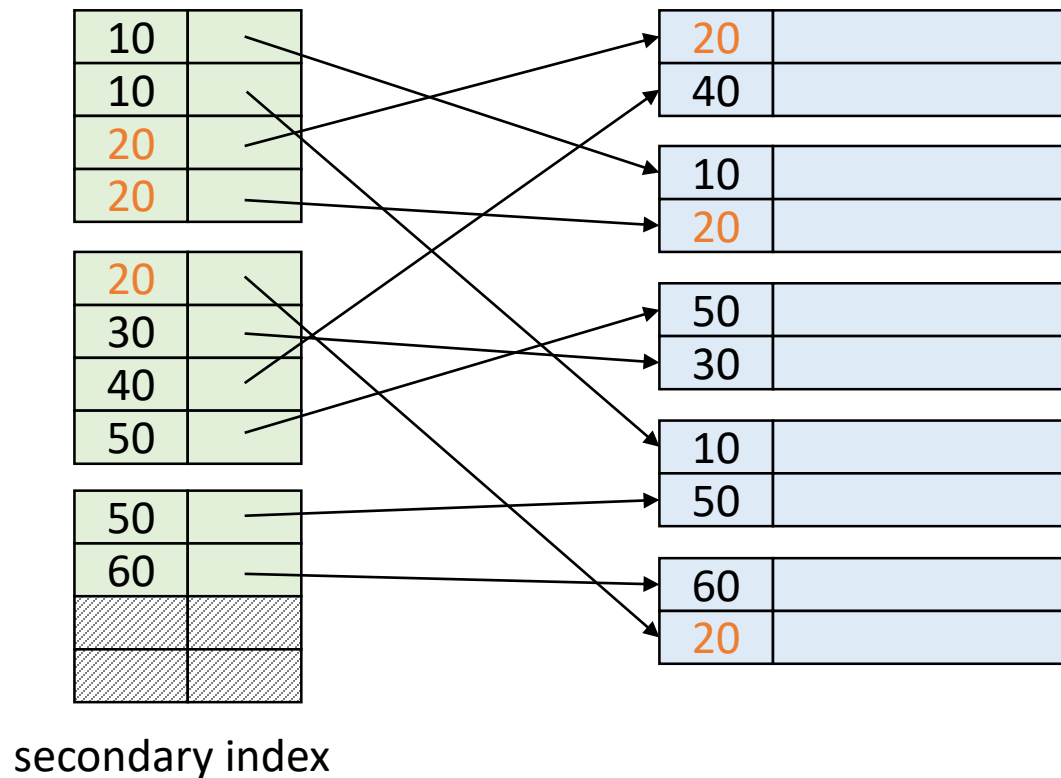
F.K.

```
SELECT title, year
FROM Movie, Studio
WHERE presidentID=zzz AND Movie.studioName = Studio.name;
```



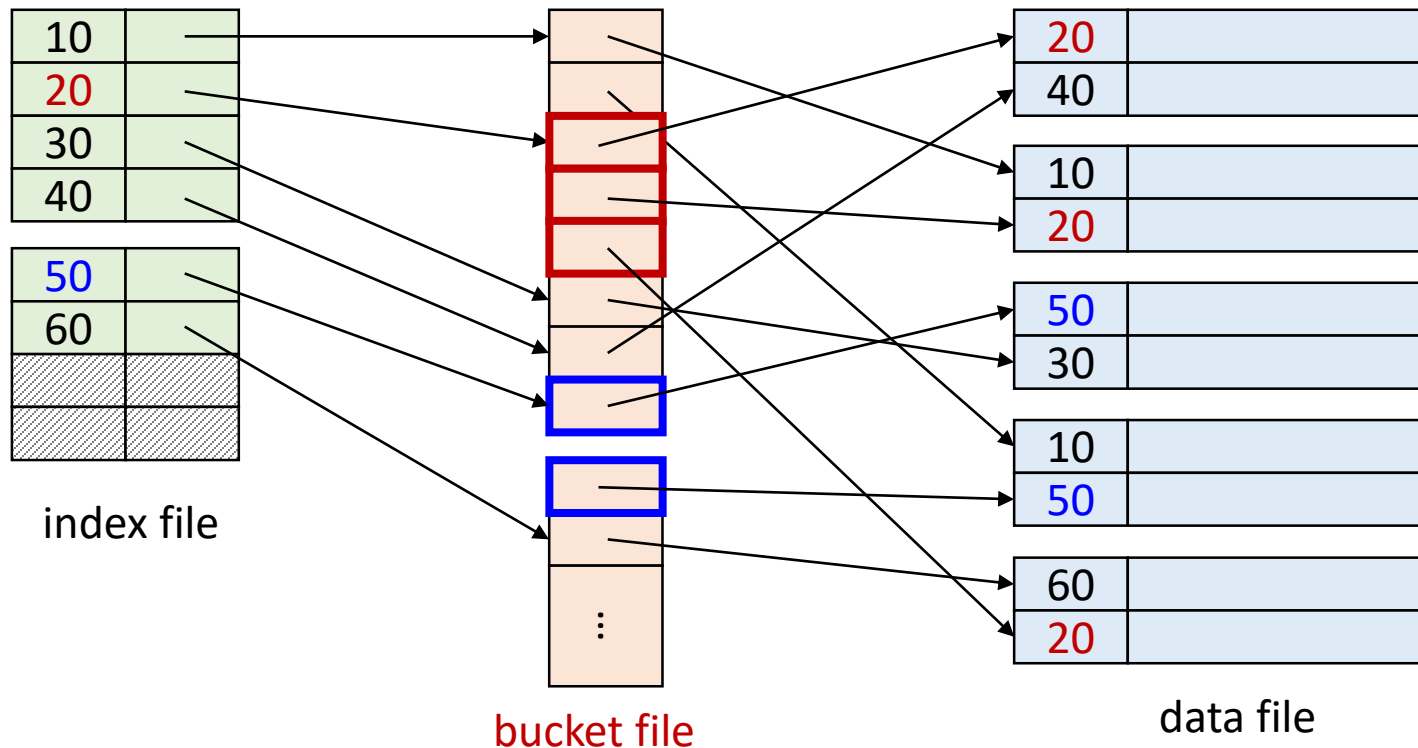
Indirection in secondary indexes (1)

- Significant amount of wastage of secondary indexes
- **Bucket file** can improve space efficiency



Indirection in secondary indexes (2)

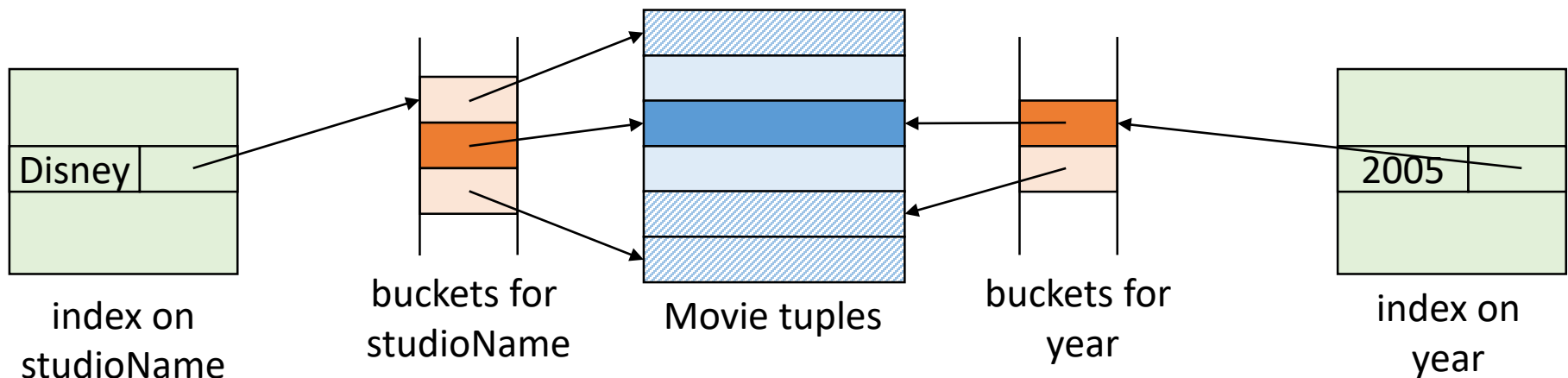
- Significant amount of wastage of secondary indexes
- Using a level of indirection (called **buckets**)
 - write a key value once for all the pointers to data records



Answering queries without looking at data records

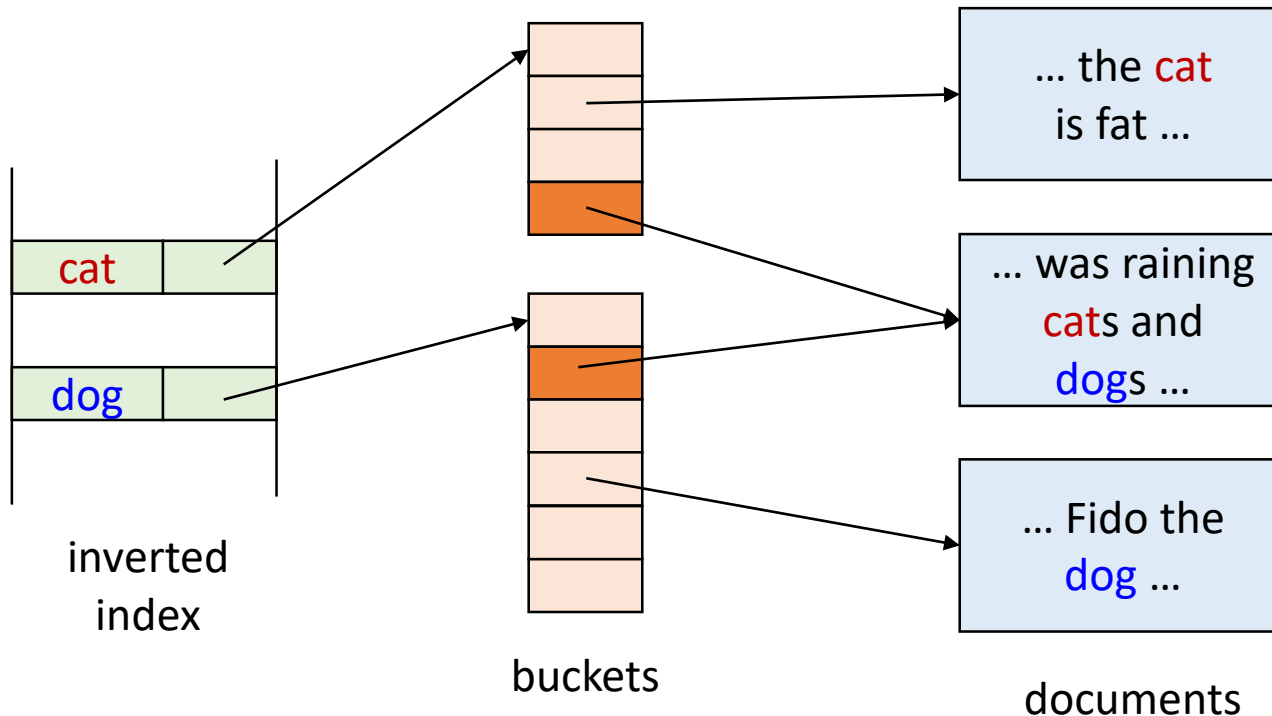
- Find the bucket pointers satisfying the conditions by intersecting sets of pointers in memory
- Retrieve only the records pointed to by the surviving pointers

```
SELECT title  
FROM Movie  
WHERE studioName='Disney' AND year=2005;
```



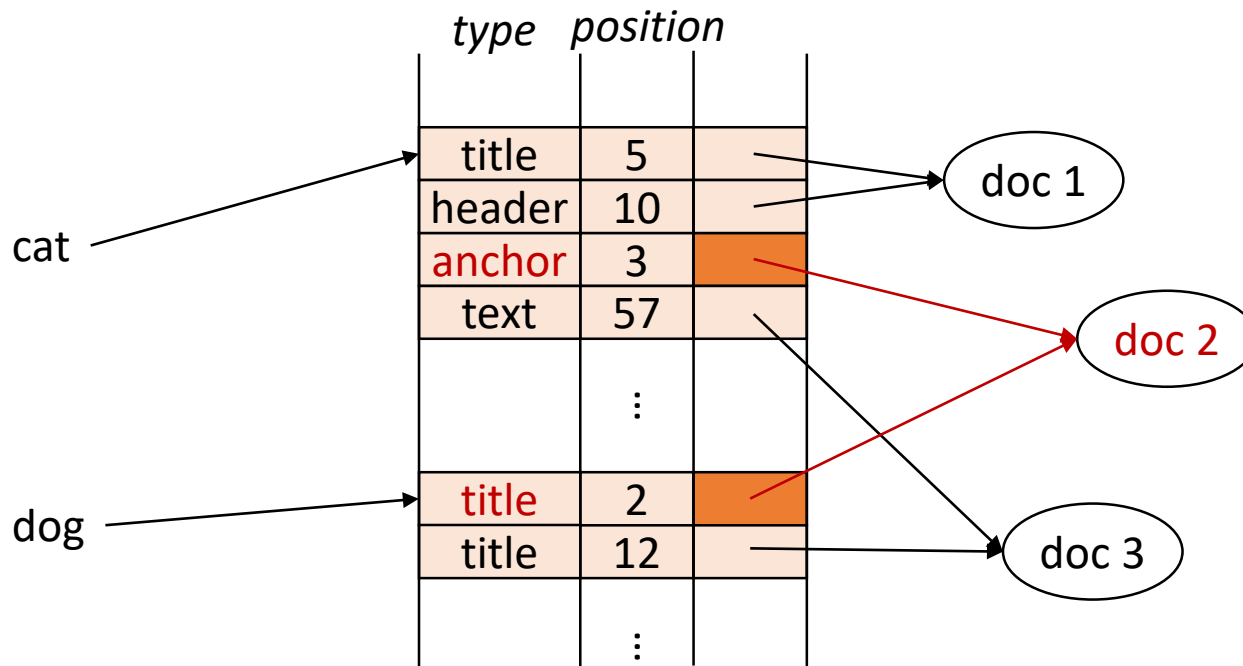
Document retrieval and inverted indexes

- Retrieval of documents containing given keywords
 - e.g., Google's keyword search



More information in the inverted index

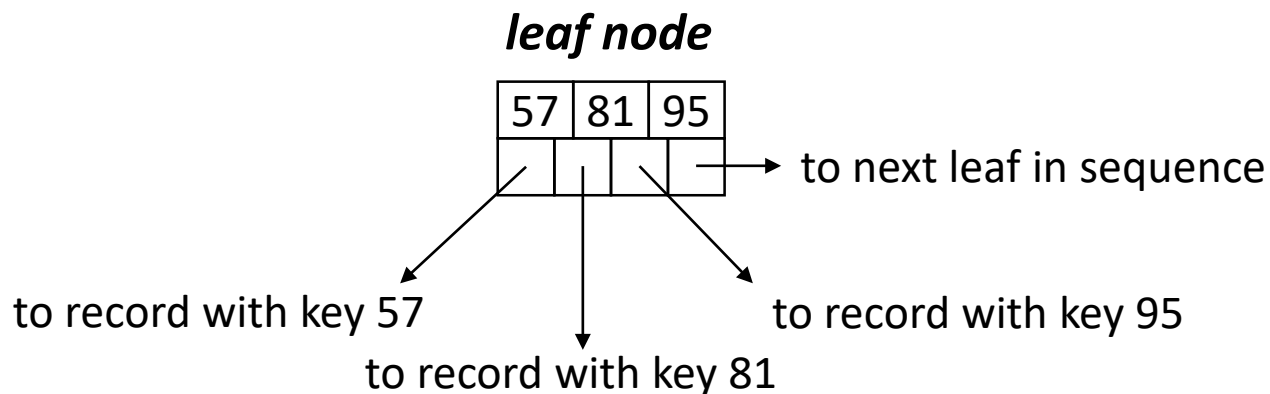
- Bucket file itself can become a collection of records
 - e.g., type of occurrence, position of occurrence



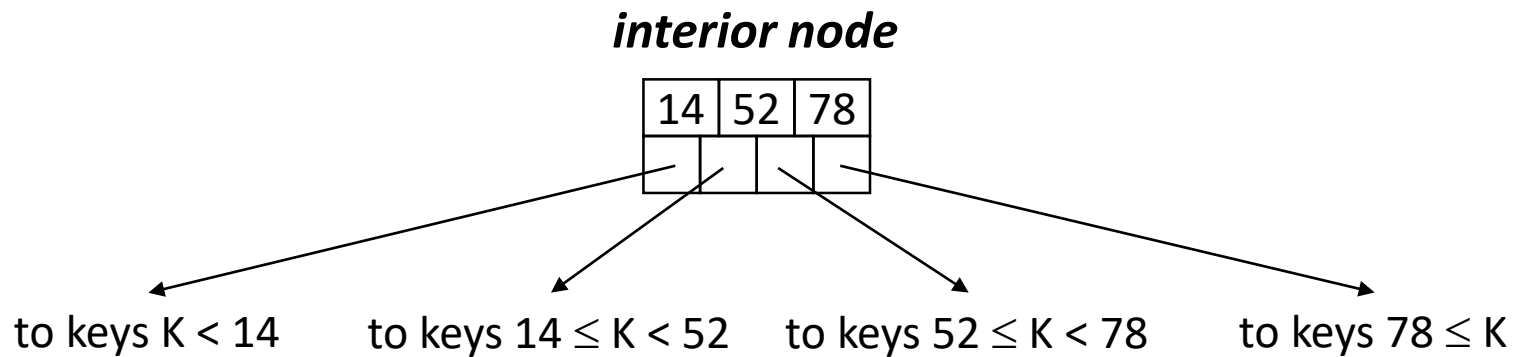
“Finding the documents where “cat” occurs at the 3rd word in the anchor, and at the same time, “dog” occurs at the second word in the title”

B-tree (known as a B+ tree)

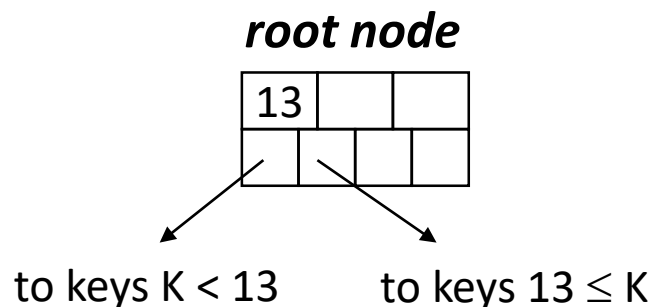
- One of the most general structures used in commercial systems
- The blocks of B-tree are organized into a balanced tree
- Each block consists of up to n search-key values and $n+1$ pointers
- e.g., block: 4096 bytes, key: 4 bytes, pointer: 8 bytes,
 $4n + 8(n+1) \leq 4096, n = 340$
- Leaf node: at least $\lfloor (n+1) / 2 \rfloor$ pointers are used to point to records



- Interior node
 - at least $\lceil (n+1)/2 \rceil$ pointers are used to point to B-tree blocks
 - all $n+1$ pointers can be used

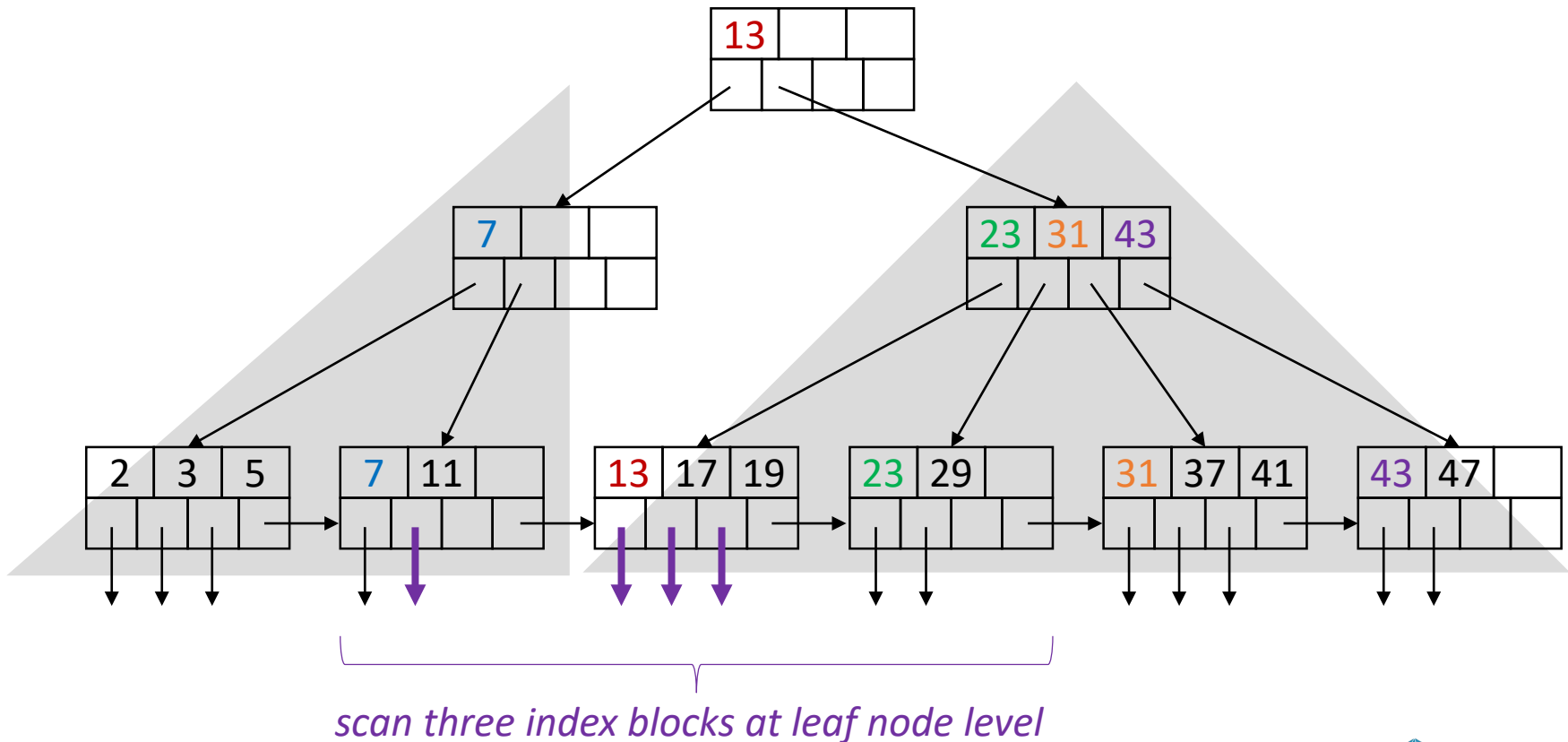


- Root node: at least two used pointers (regardless of n)
 - exception: data file has only one record



Example of B-tree

- $n=3$, $\lfloor (n+1) / 2 \rfloor = 2$ (for leaf), $\lceil (n+1)/2 \rceil = 2$ (for interior)

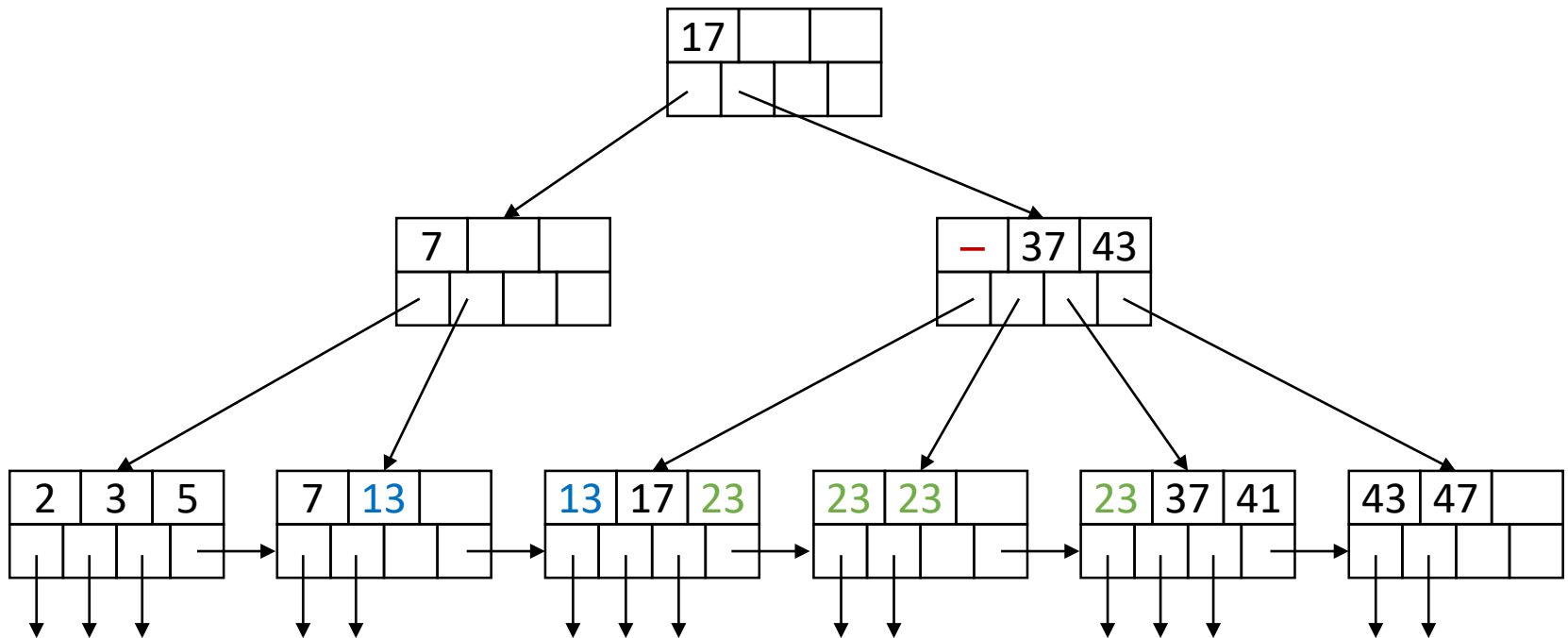


Applications of B-trees

- Example 1: B-tree on primary key and dense index
 - one key-pointer pair in a leaf for every record of data file
 - data file may or may not be sorted by primary key
- Example 2: B-tree on primary key and sparse index
 - data file is sorted by its primary key
 - one key-pointer pair at a leaf for each block of data file
- Example 3: B-tree on non-primary key
 - if data file is sorted by an attribute that is not a key
 - the attribute is the search key for B-tree
 - one key-pointer pair for the first of the records having the same values
 - but, if data file is not sorted,
 - B-tree must maintain every key-pointer pair for every record having the same key value

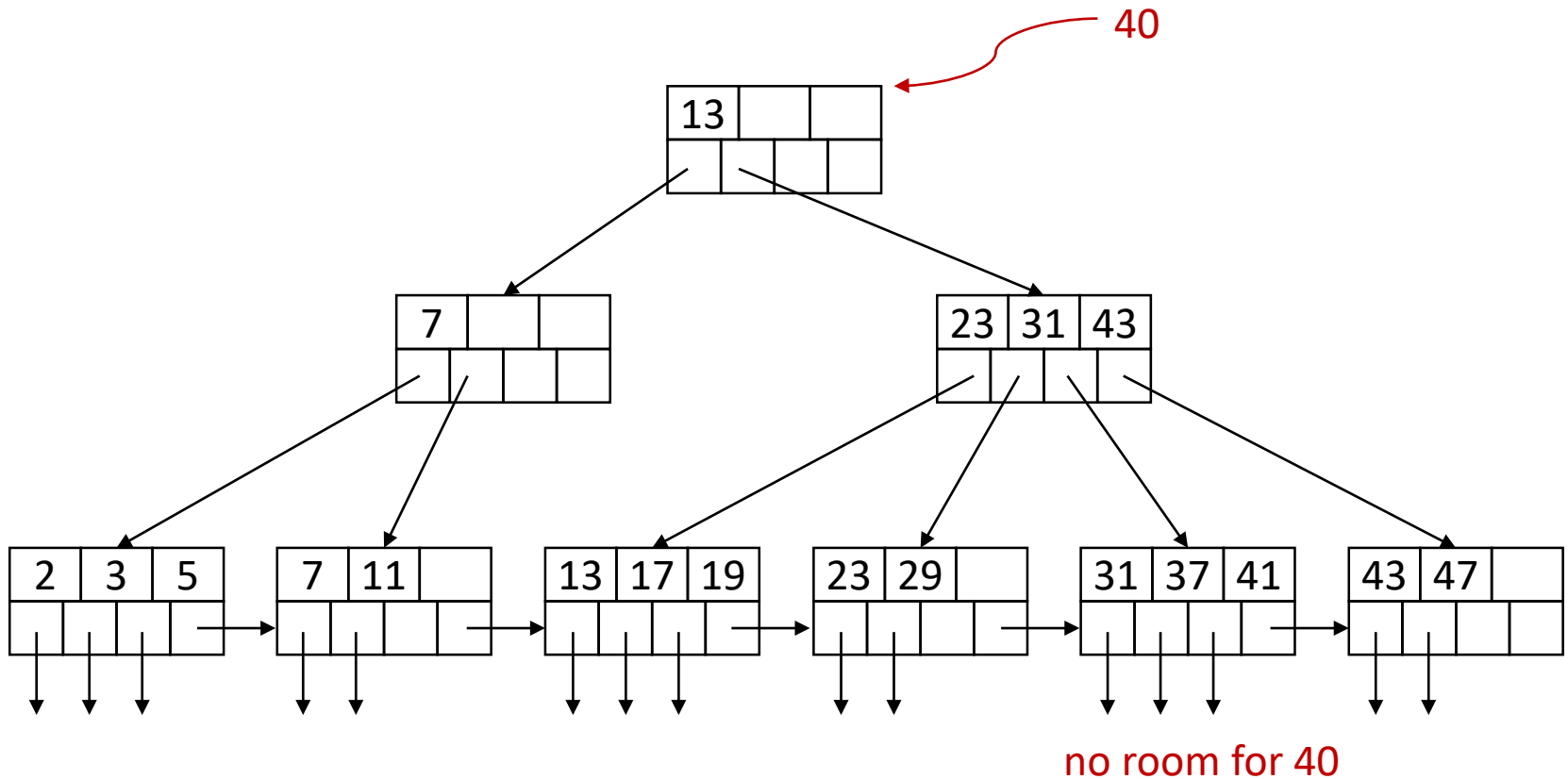
B-tree variant

- Allowing multiple occurrences of the search key
 - key in the interior node can be taken to be null



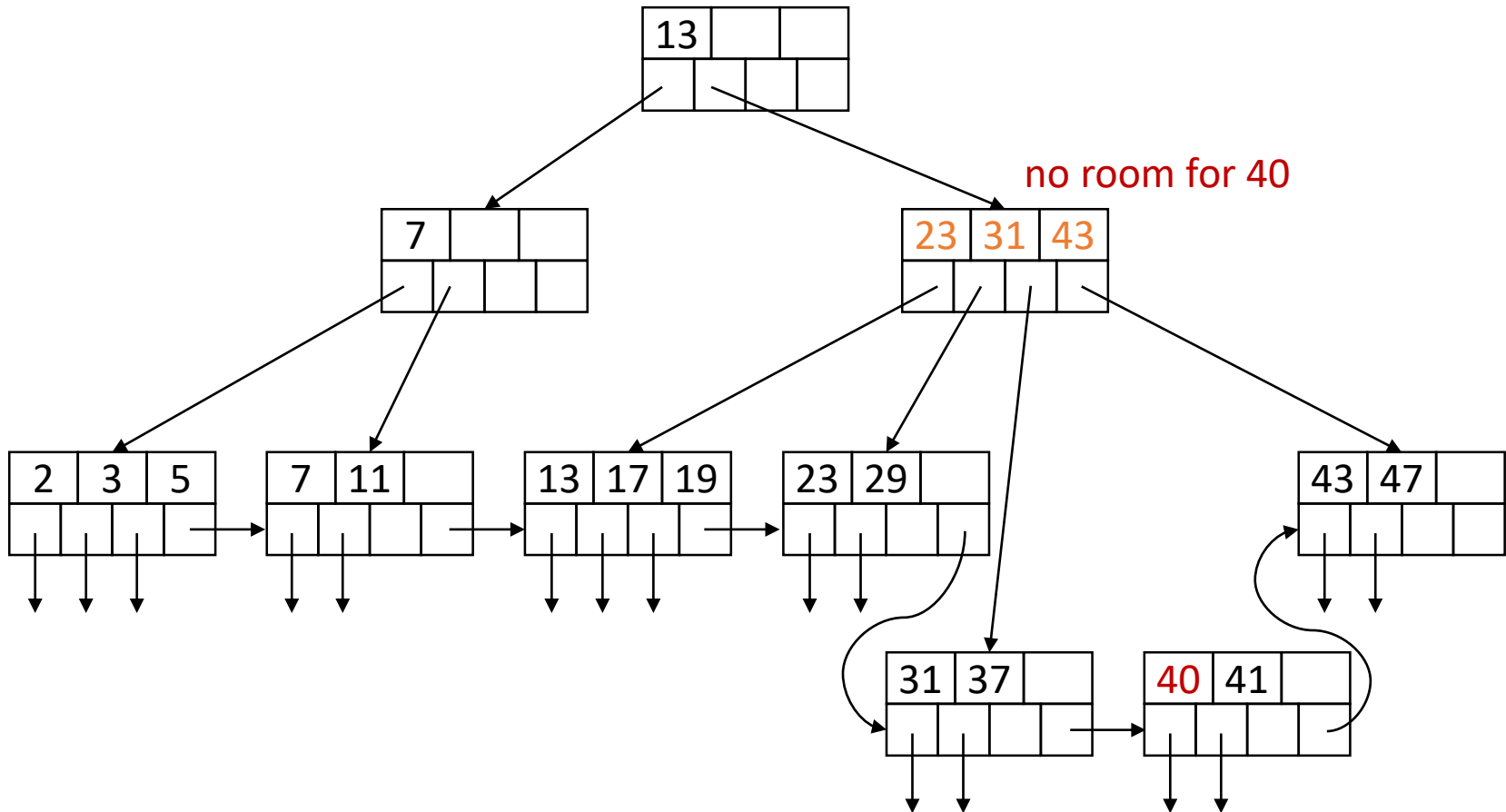
Insertion into B-trees (1)

- Recursively apply the below strategy
 - if there is room, insert it
 - if not, split the node and continue up to tree



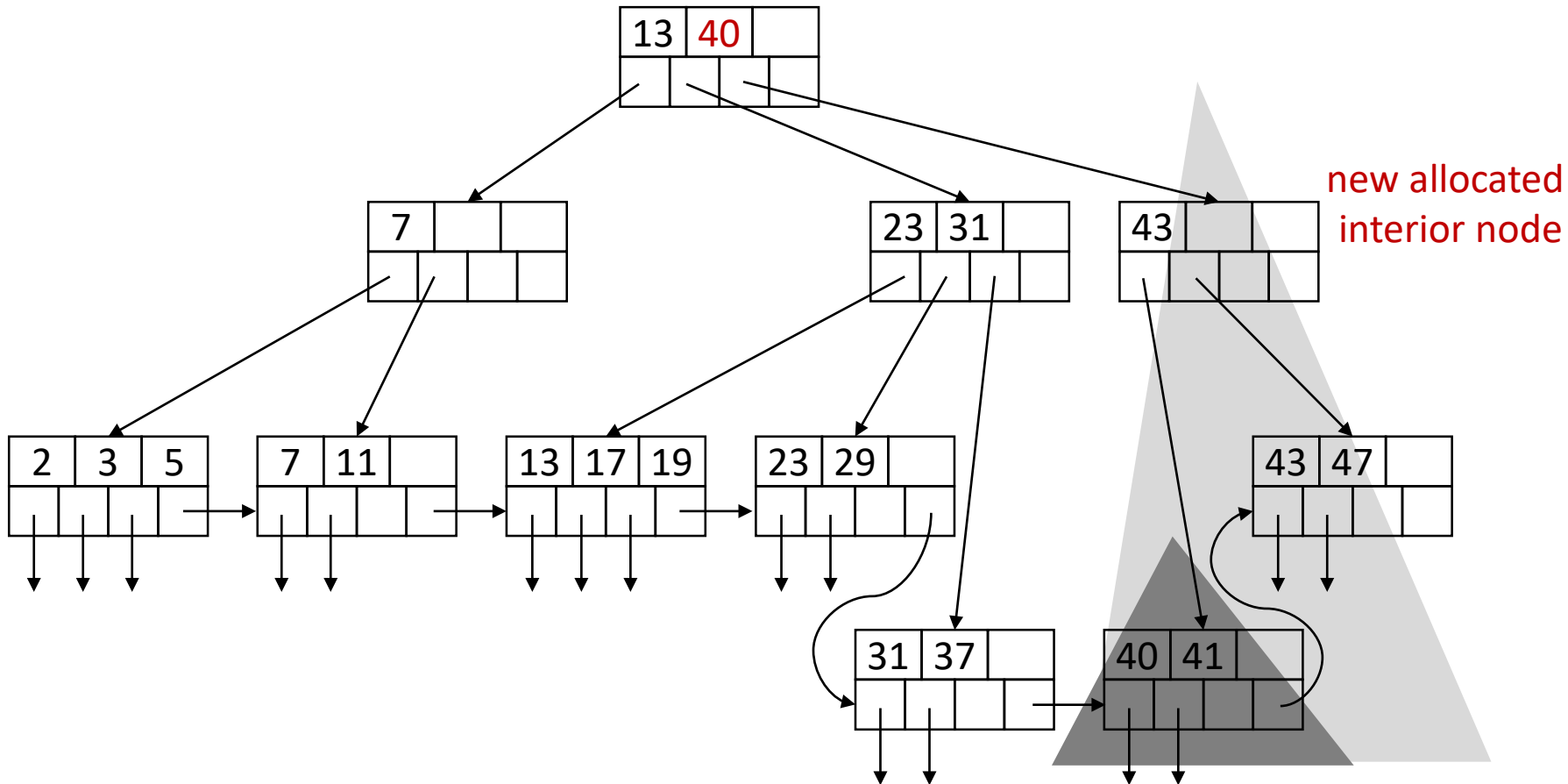
Insertion into B-trees (2)

- Recursively apply the below strategy
 - if there is room, insert it
 - if not, split the node and continue up to tree



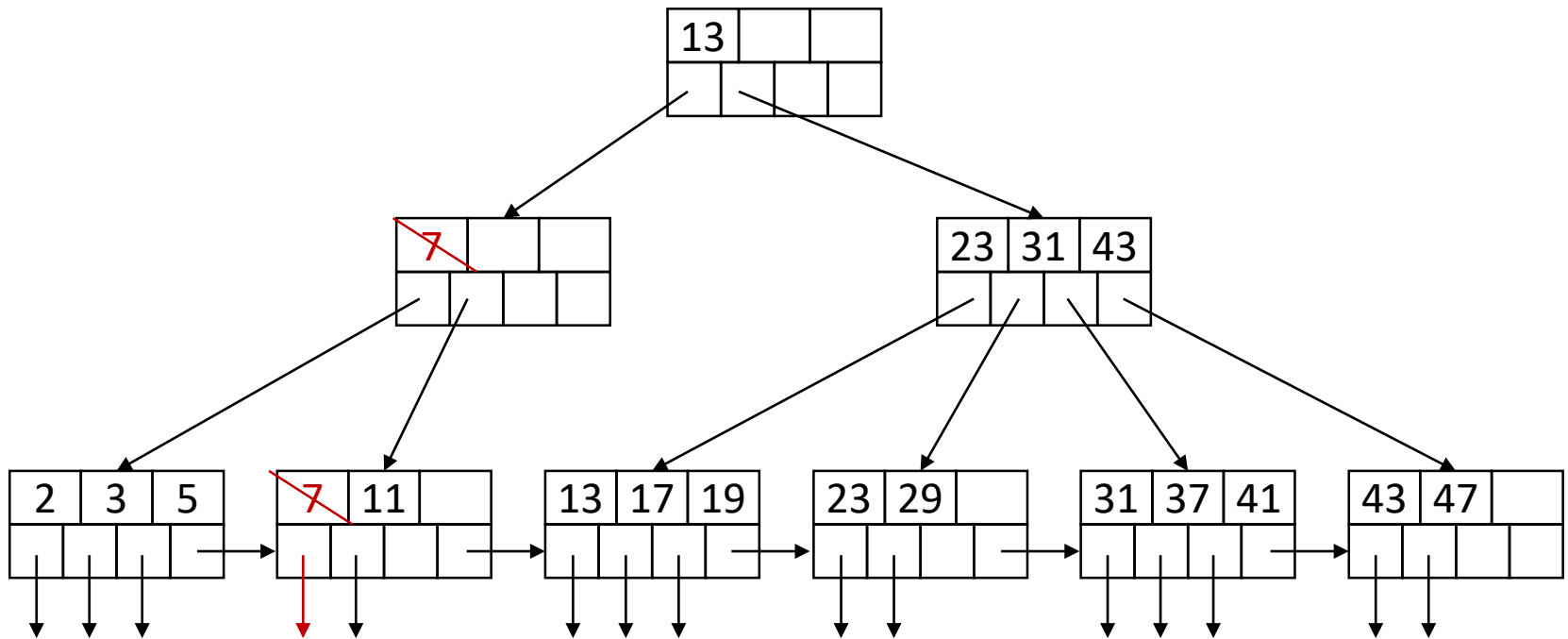
Insertion into B-trees (3)

- Recursively apply the below strategy
 - if there is room, insert it
 - if not, split the node and continue up to tree



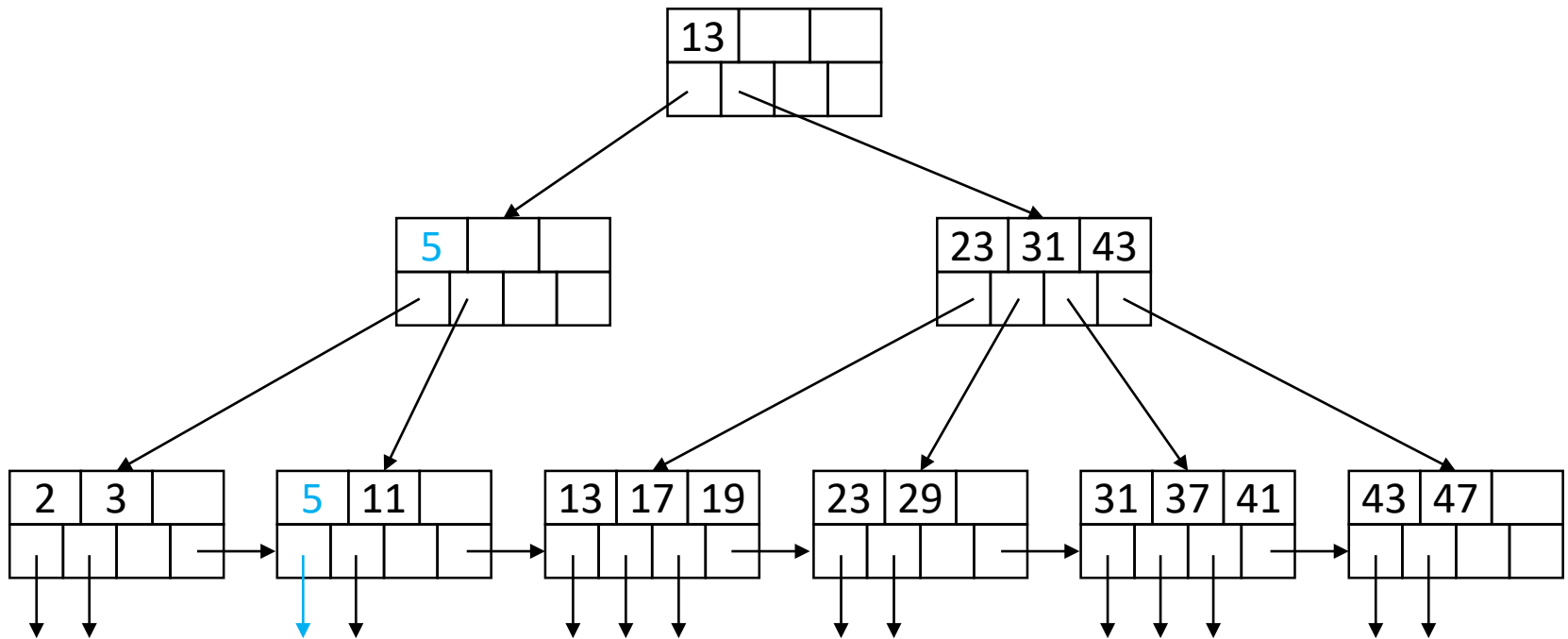
Deletion from B-trees (1)

- Nodes still must have the minimum number of keys and pointers after deletion



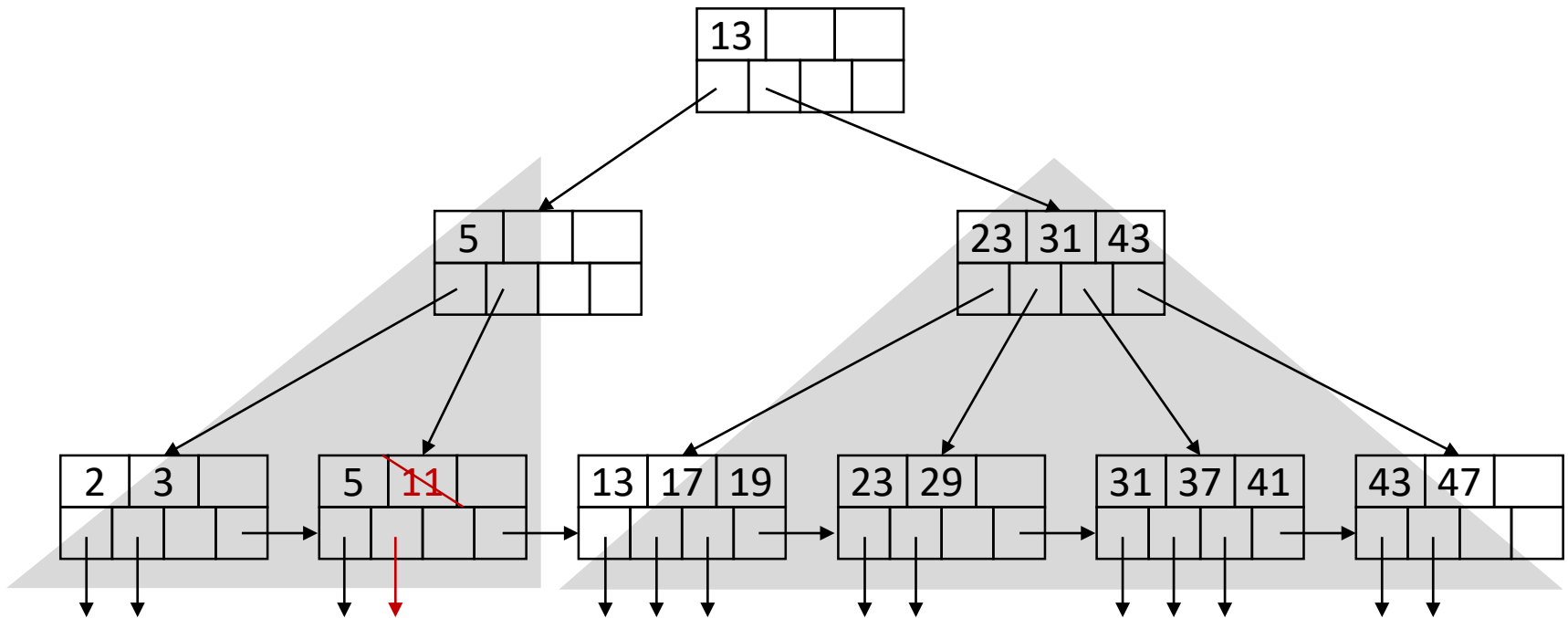
Deletion from B-trees (2)

- Nodes still must have the minimum number of keys and pointers after deletion



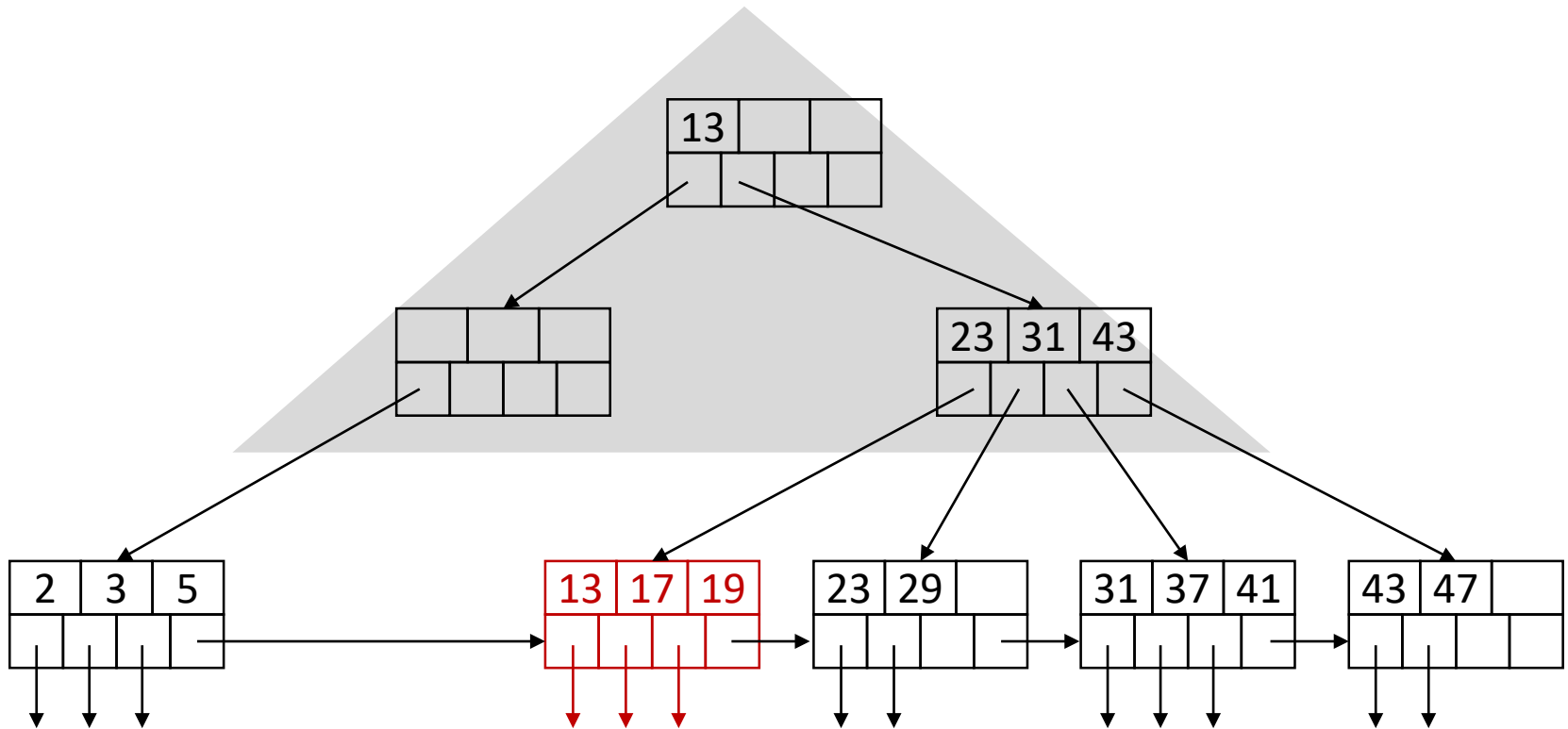
Deletion from B-trees (3)

- If there is no sibling to the right from which to take a key, **merge** the leaf with its sibling
- Adjust the keys (delete a key and pointer) at the parent



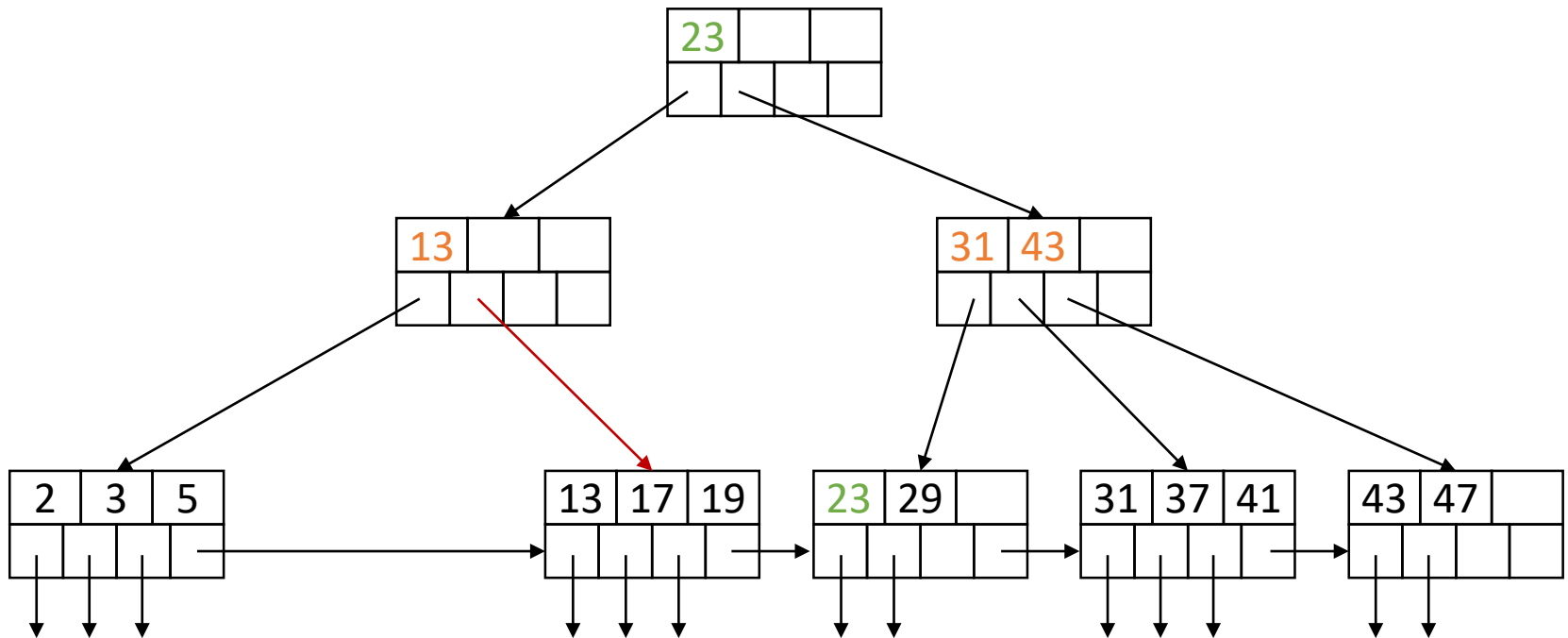
Deletion from B-trees (4)

- If the parent is not full enough, recursively apply the deletion at the parent



Deletion from B-trees (5)

- e.g., the pointer the leaf with keys {13, 17, 19} is moved from the second child of the root to the first child

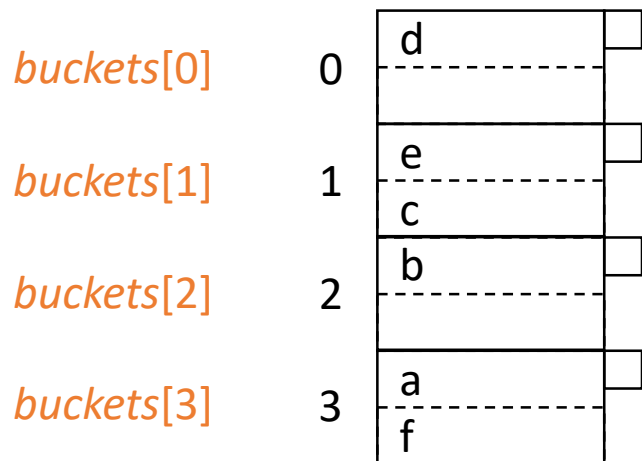


Efficiency of B-trees

- Lookup / insertion / deletion of records using very few disk I/O's
 - lookup: (# of levels + 1) disk I/O's
 - splitting and merging of blocks are rare events
 - they are also almost always limited to the leaves
 - insertion/deletion: (# of levels + 2) disk I/O's
- # of levels: depending on fan-out
 - e.g., node can have 255 children, $255^3 = 16.6$ million records can be accommodated by a 3-level B-tree
 - e.g., 4-level: 4.2 billion records, 5-level: 1 trillion records
 - root block is usually buffered in main memory

Secondary-storage hash tables

- Hash function h
 - take a search key (i.e., hash key) K as an argument
 - compute from it an integer in the range $[0, B-1]$
 - B : number of buckets
- **Bucket array** consists of blocks
 - a record having K is stored in the block for $\text{buckets}[h(K)]$
 - a chain of overflow blocks can be added to the bucket



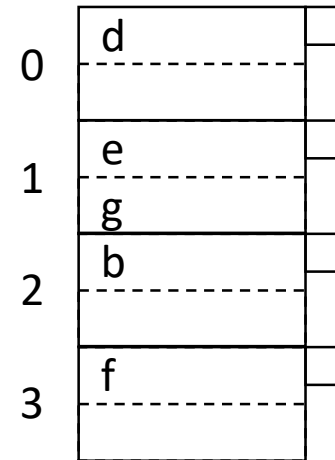
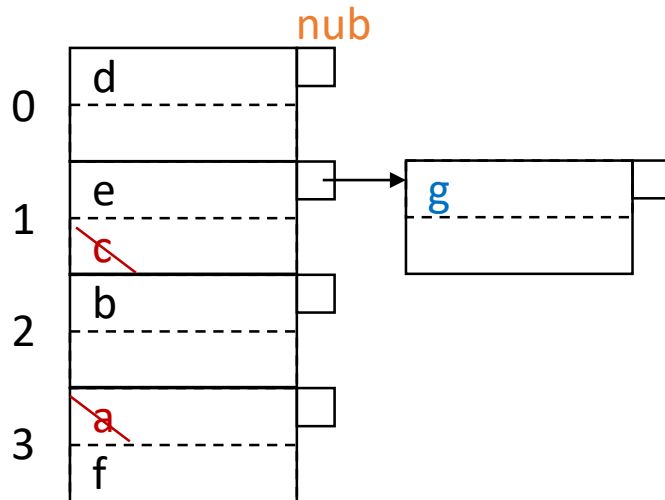
$B=4$

two records per block

$h(d) = 0$

Insertion and deletion

- e.g., insert g, and $h(g) = 1$
- e.g., delete c and a

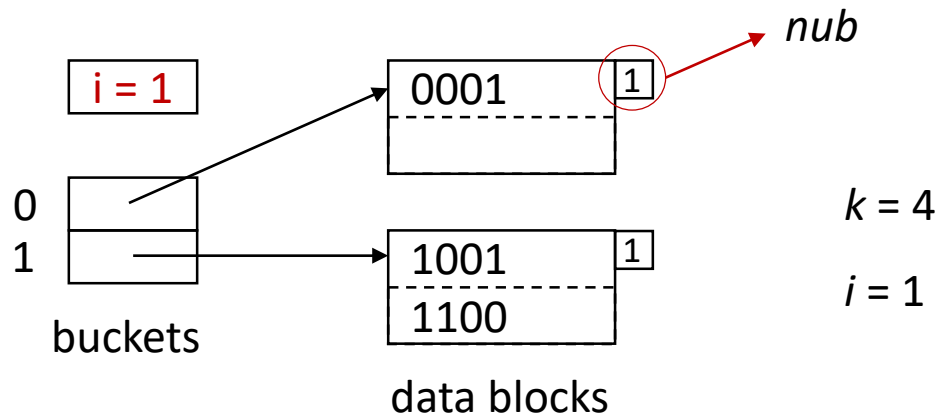


Efficiency of hash table indexes

- Lookup : 1 disk I/O's
 - Insertion / deletion : 2 disk I/O's
- } if there are enough # of buckets
- Static hash tables
 - if the file grows, there are eventually many blocks in the chain
 - we need to search long lists of blocks (at least one disk I/O per block)
 - it is important to keep # of blocks per bucket low
 - Dynamic hash tables
 - B varies such that there is about one block per bucket
 - extensible hashing
 - linear hashing

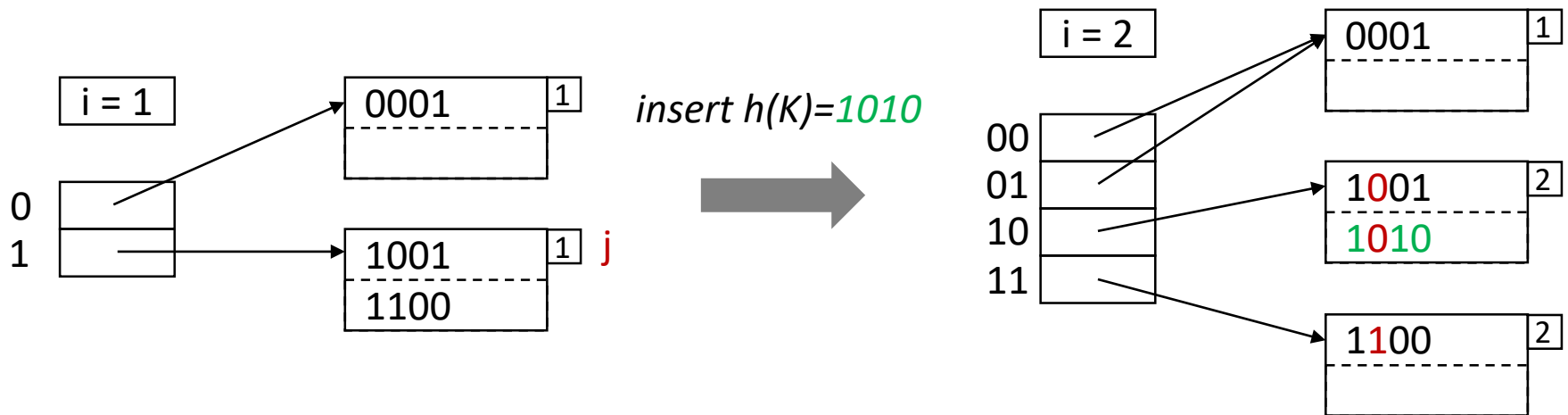
Extensible hash tables ($B \rightarrow 2B$)

- Indirection for the buckets
 - array of pointers to blocks (instead of array holding data blocks)
- Array of pointers can grow
 - length is always a power of 2 (# of buckets doubles)
- Certain buckets can share a block
 - if the total # of records in those buckets can fit in the block
- Hash function computes using only i bits for each key (k bits)
 - bucket array will have 2^i entries



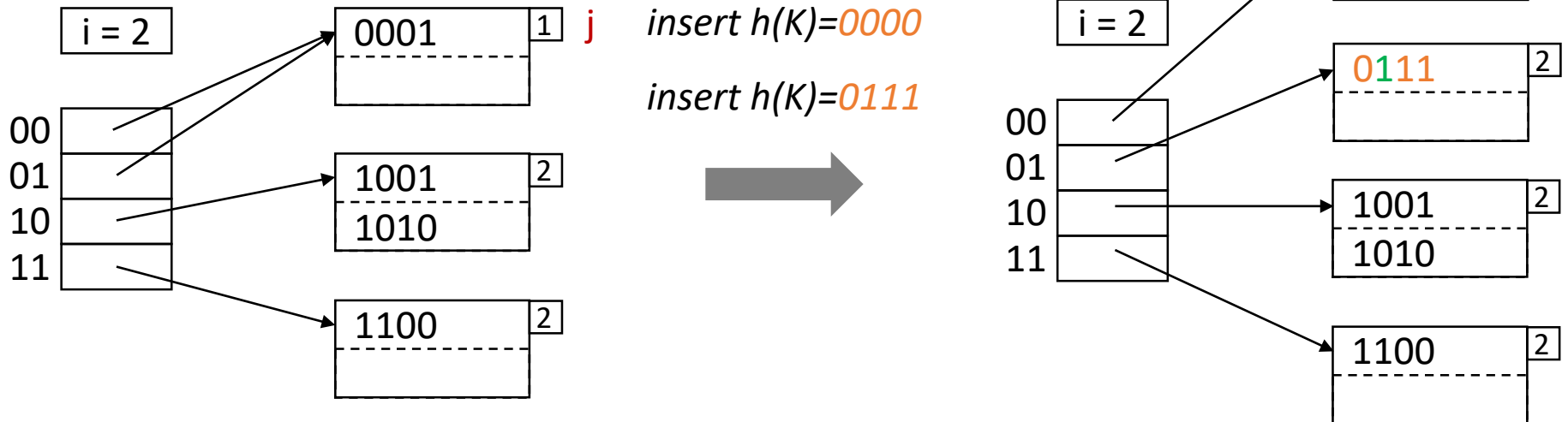
Insertion into extensible hash tables

- if no room in the bucket (j : value in the nub of the bucket)
- Case $j = i$
 - increase i by 1
 - double the length of the bucket array
 - split the block



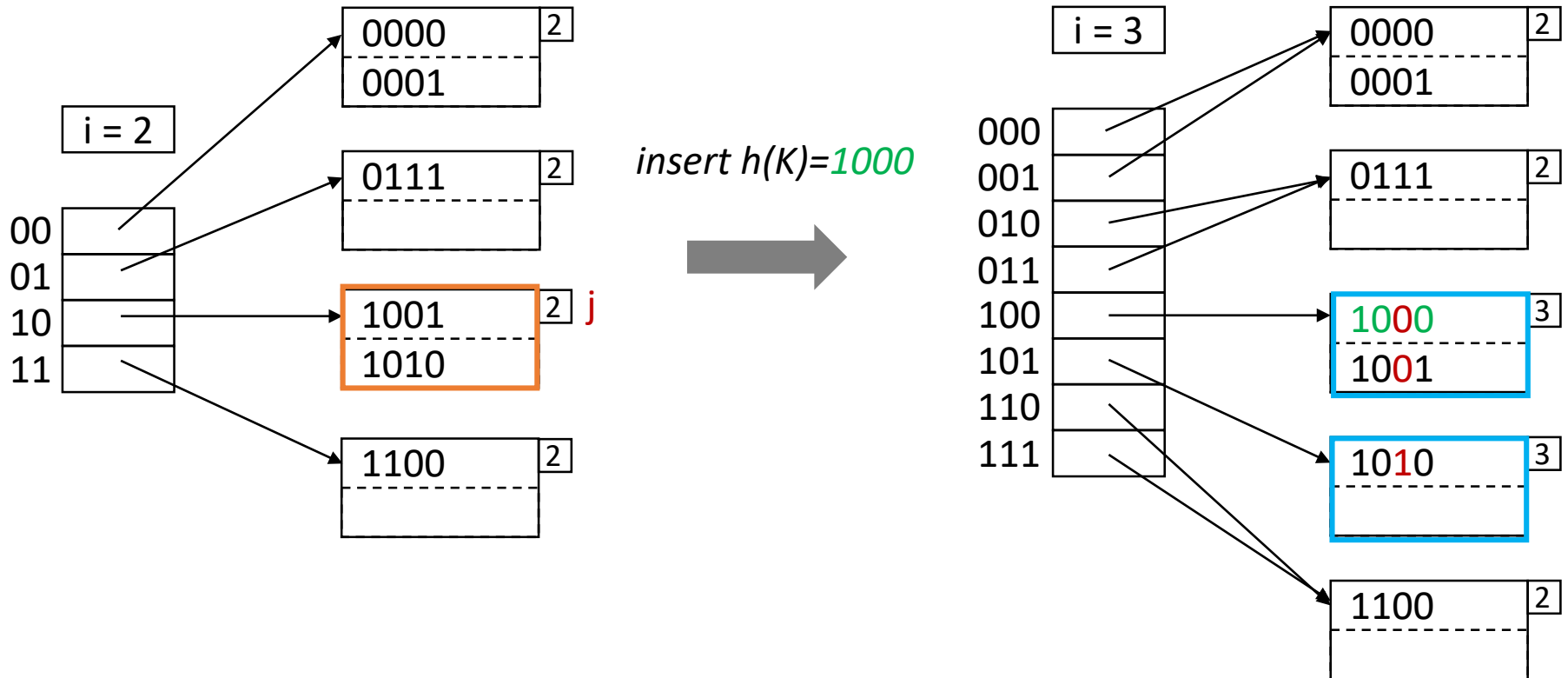
- Case $j < i$

- split the block
- distribute records in the blocks based on the value of $(j+1)$ th bit
- put $(j+1)$ in the nubs of the blocks
- adjust the pointers in the bucket array



- Case $j = i$

- increase i by 1
- double the length of the bucket array
- split the block



Pros and cons of extensible hash tables

- Pros

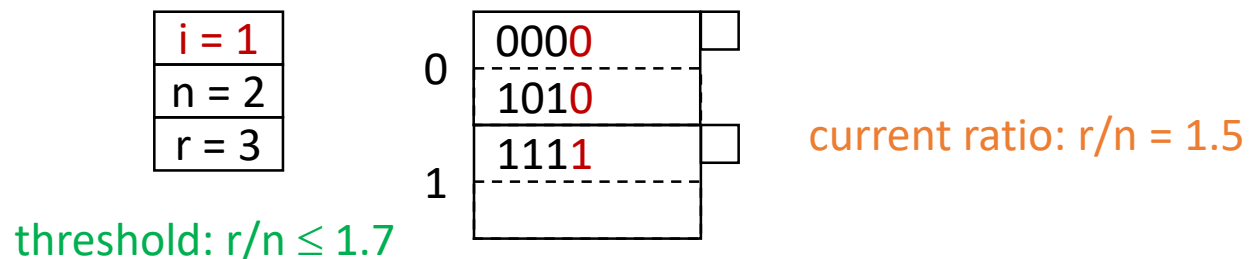
- never need to search more than one data block
- no additional disk I/O, if the bucket array fits in main memory

- Cons

- substantial amount of work to be done for doubling the bucket array (in particular, when i is large)
- doubled bucket array may no longer fit in main memory, then a system might suddenly start using many more disk I/O's
- if the distribution of keys is skew, we need to use too many i bits and too large bucket array, although # of records is small

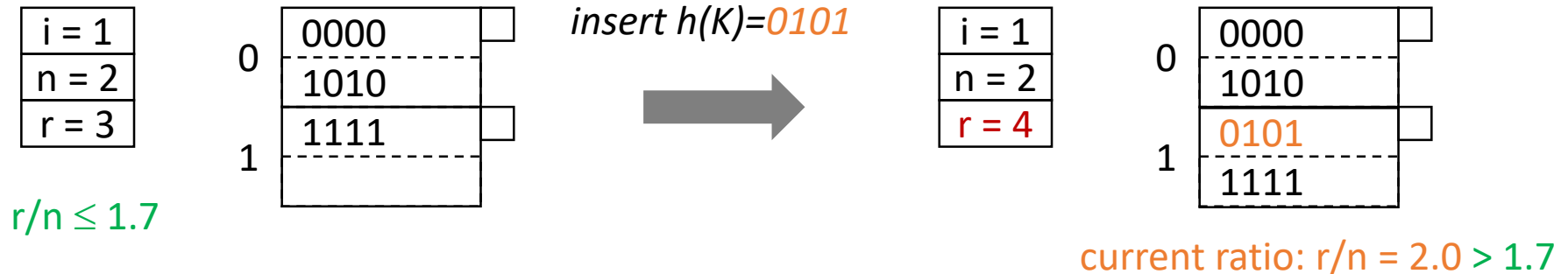
Linear hash tables

- # of buckets grows more slowly
- Average # of records per bucket is a fixed fraction (load factor)
 - r : the current # of records in the hash table
 - n : the current # of buckets
 - ratio r/n is limited (e.g., up to 1.7 records per bucket)
- Overflow blocks are permitted when blocks CANNOT be split
- Only low-order $i = \lceil \log_2 n \rceil$ bits are used for n buckets



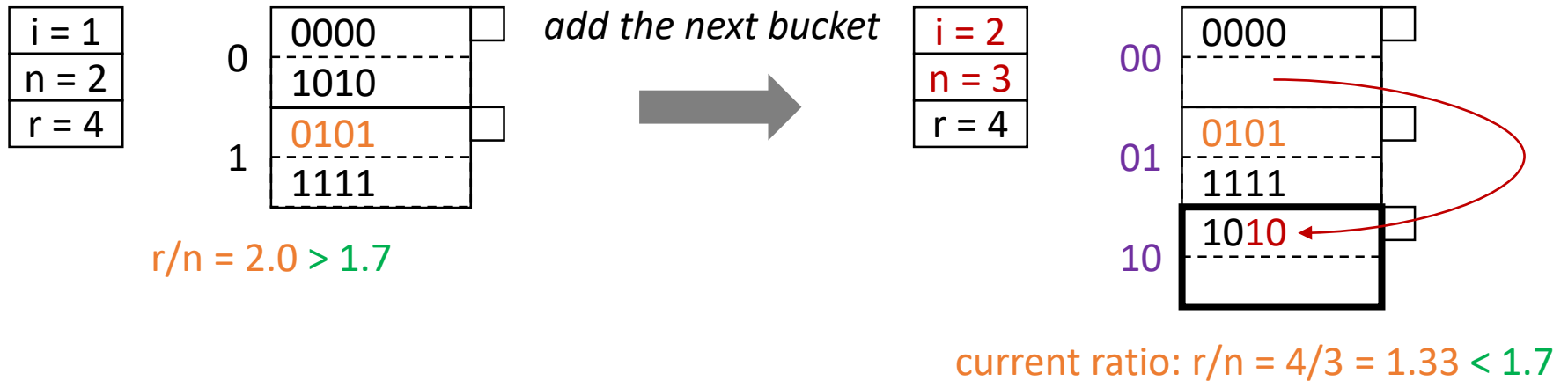
Inserting a key K into a linear hash table (1)

- ① If $h(K)=m$, and $m < n$, put the record in bucket m
- ② ratio r/n is too high, add the next bucket to the table
 - e.g., $4/2 = 2$ is too high, must raise n from 2 to 3
 - e.g., since $i = \lceil \log_2 3 \rceil = 2$, the indexes of buckets are changed



Inserting a key K into a linear hash table (2)

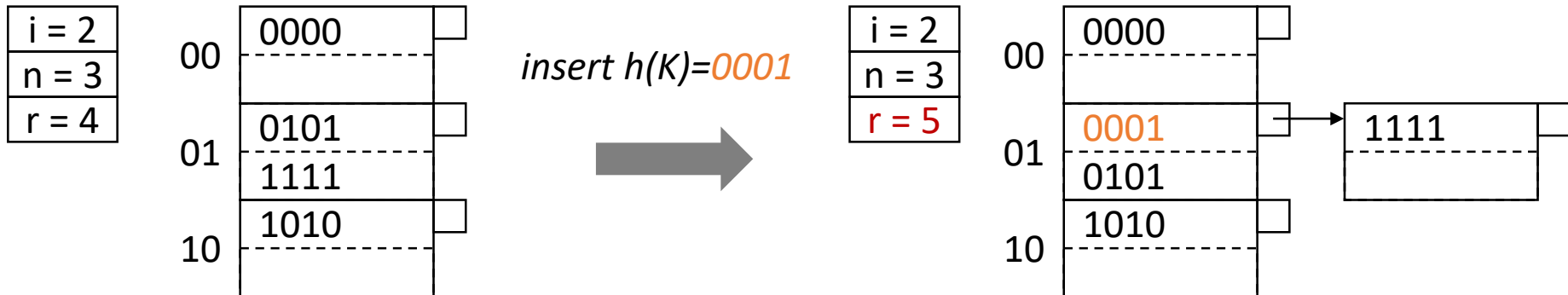
- ① If $h(K)=m$, and $m < n$, put the record in bucket m
- ② ratio r/n is too high, add the next bucket to the table
 - e.g., $4/2 = 2$ is too high, must raise n from 2 to 3
 - e.g., since $i = \lceil \log_2 3 \rceil = 2$, the indexes of buckets are changed



Inserting a key K into a linear hash table (3)

③ If there is no room in bucket, create an overflow block

- e.g., $m=1$, $n=3$, $m < n$, no room, so create an overflow block



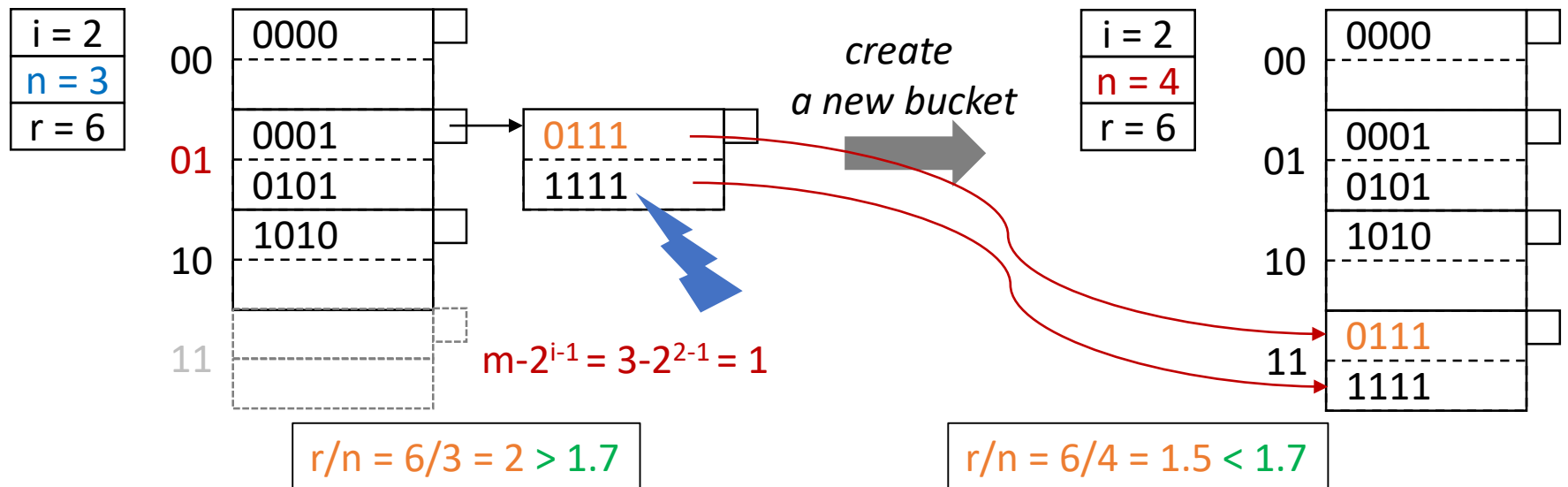
current ratio: $r/n = 5/3 = 1.67 < 1.7$

Inserting a key K into a linear hash table (4)

④ If $m \geq n$, put the record in bucket $m - 2^{i-1}$

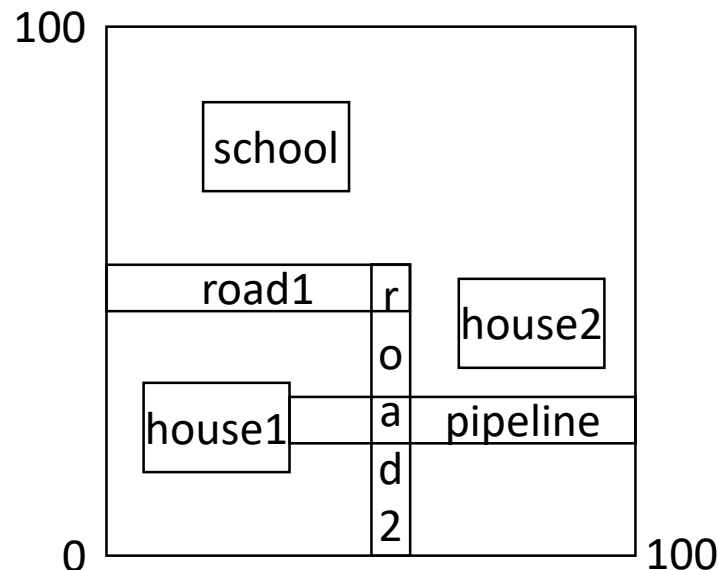
• e.g., insert 0111

- bucket 11 does not yet exist ($m = 3, n = 3$, so $m \geq n$)
- redirect the record to bucket 01 (can fit in the overflow block)
- ratio r/n is too high, and so, create a new bucket
- move two records from the overflow bucket to the new bucket
- delete the overflow bucket



Multidimensional data space

- One dimensional indexes
 - retrieve records that match a given **single search key value**
 - **single linear order** for the keys (for tree-based indexes)
- Many applications requiring multidimensional data space
 - 2-dimension: **Geographic Information System (GIS)**, IC design
 - 3-dimension: medical data (MRI), computer graphics/games



Queries of GIS

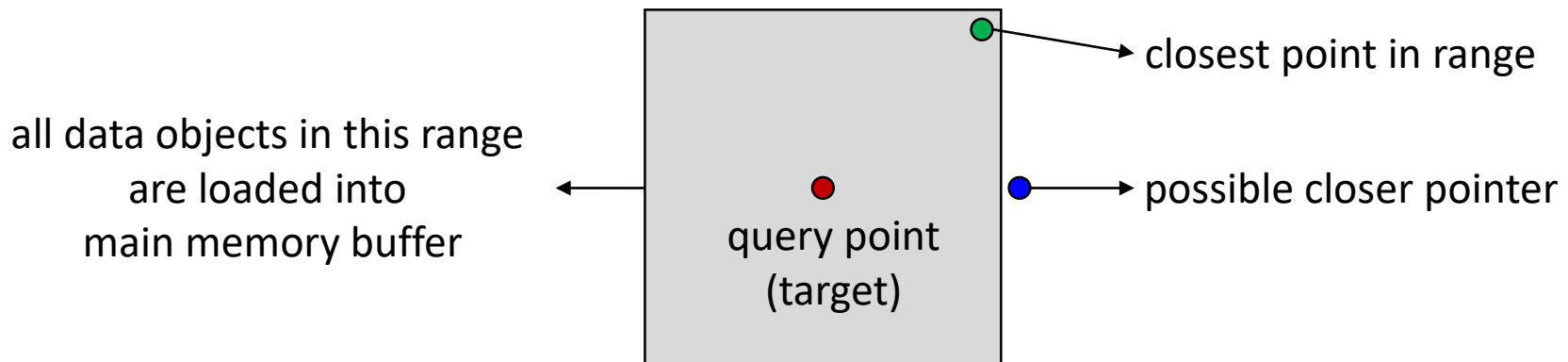
- **Partial match** queries
 - find all points matching the values specified in one or more dimensions (NOT all dimensions)
- **Range** queries
 - find the set of points within the range specified in one or more dimensions
- **Nearest-neighbor** queries
 - find the closest point(s) for a given point
- **Where-am-I** queries
 - find where a given point is located, and in which shape the point is located

Range queries using conventional indexes

- Using two B-trees for two dimensions x and y
 - build secondary indexes on x and y
 - get pointers to all the records in the range for x and y
 - intersect both sets of pointers
 - total # of disk I/O's: (# of nodes examined in both B-trees) + (# of data blocks to retrieve all the matching records)
 - assumption: the pointers fit in memory, root node of B-tree is cached
- e.g., $x=[0,1000]$, $y=[0,1000]$, $n=1,000,000$
 - 100 records per block, avg. 200 key-pointer pairs in B-tree leaf
 - query: $[450 \leq x \leq 550] \ \& \ [450 \leq y \leq 550]$
 - approximately 10,000 result pointers ($\approx 100,000 \cap 100,000$)
 - # of disk I/O's for scanning B-trees: $(500 + 1) + (500 + 1) = 1002$
 - # of disk I/O's for retrieving records: 10000 ($\approx$ all data blocks!)
 - # of total disk I/O's : 11002

Nearest-neighbor queries using conventional indexes

- Basic method
 - pick a range in each dimension, asking the range query
 - select the point closest to the target within that range
- Two possible results
 - no point within the selected range → repeat with a larger range
 - the selected point might not be the closest point → increase the range once more and retrieve all points in the larger range

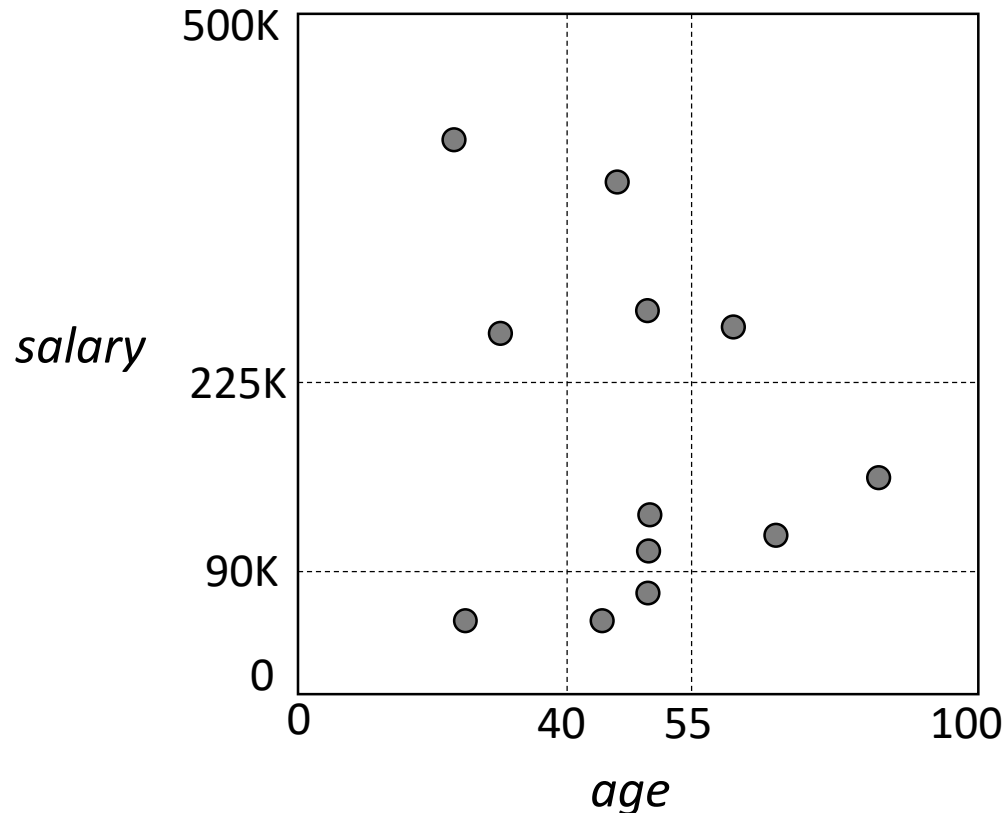


Categories of multidimensional indexes

- Hash-table-like approach
 - limit our search to a subset of the buckets
 - c.f., exactly one bucket in one-dimensional indexes
- Tree-like approach (giving up at least one of the below important properties of B-trees)
 - balance of the tree (all leaves are at the same level)
 - correspondence between tree nodes and disk blocks
 - speed of modification to data

Hash structures (1): **grid file**

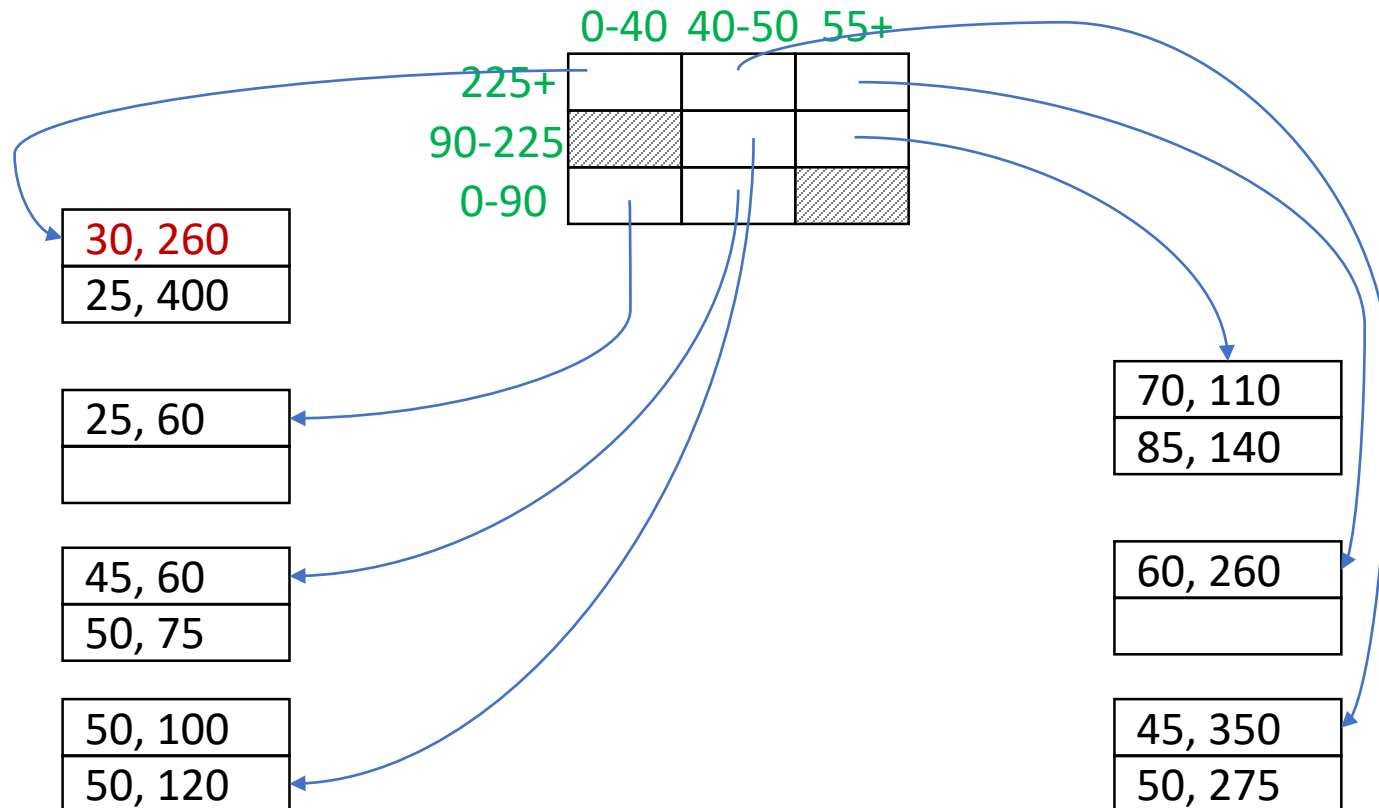
- **Grid lines** partition the space into **stripes** in each dimension
 - # of grid lines in different dimensions may vary
 - usually different spacings between adjacent grid lines



Q: who buys gold jewelry?

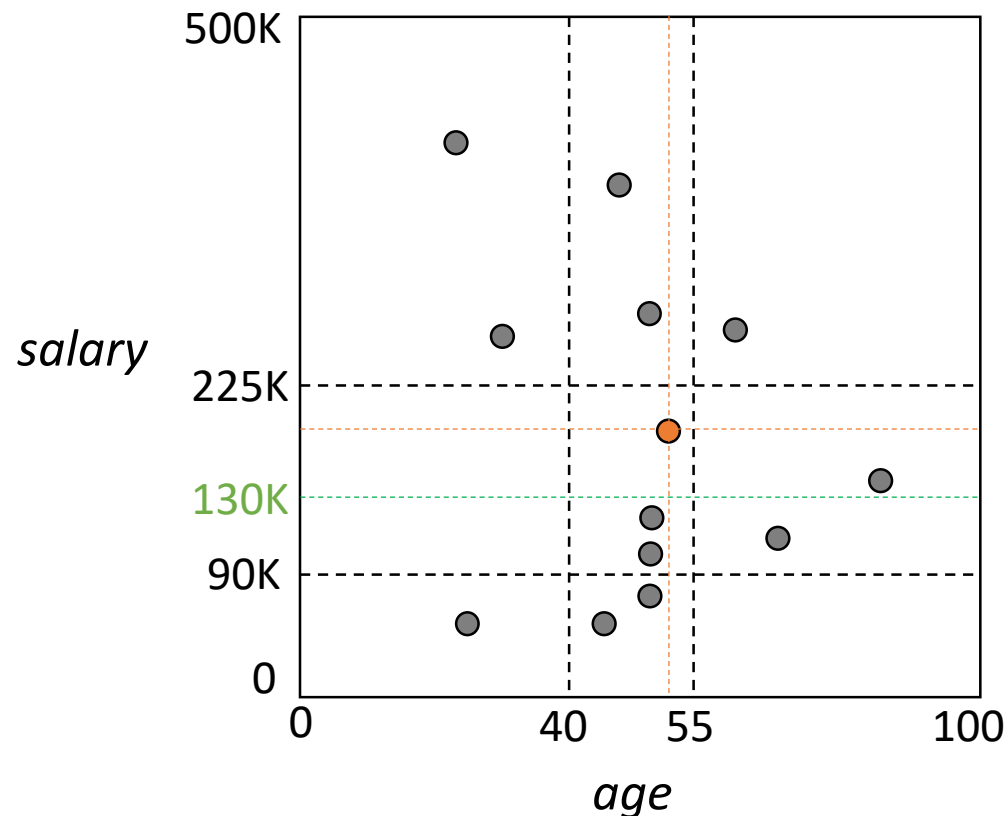
Grid file: lookup operation

- Each region can be thought as a bucket of a hash table



Grid file: insertion operation

- Two approaches when there is **no room** in the bucket
 - add overflow blocks to the buckets
 - reorganize the structure by adding or moving the grid lines

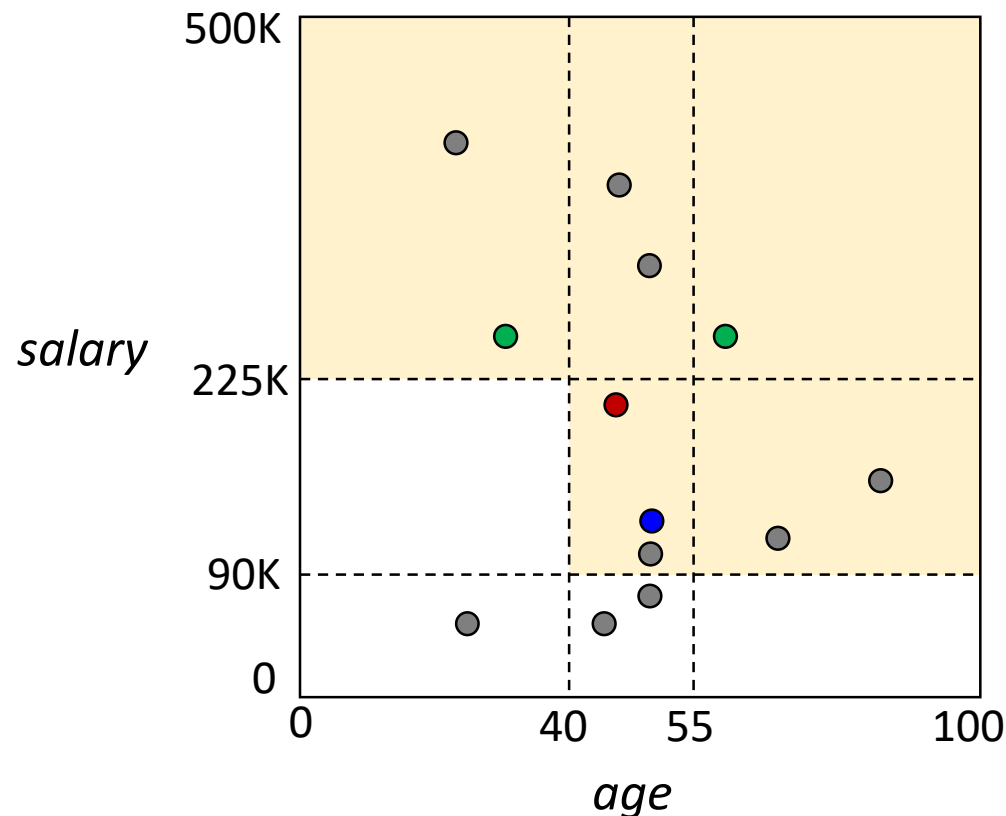


(52, \$200K)

Performance of grid files

- Case 1: the distribution of data is **skewed**
 - e.g., all points along the diagonal due to high correlation between age and salary
 - # of buckets grows exponentially with # of dimensions
- Case 2: data is **well distributed** (bucket matrix in memory)
 - **lookup of specific points**: what is necessary to read the bucket
 - **partial-match queries**: all the buckets in a row or column
 - e.g., find all customers aged 50
 - **range queries**: all the buckets to cover the range
 - e.g., find all customers aged 35-45 with a salary of 50-100 (4 buckets)
 - **nearest-neighbor queries**: adjacent buckets as well for no FN
 - e.g., $P = (45, 200)$, the closest point in bucket $(50, 120)$ (distance: 80.2)
 $(30, 260)$ and $(60, 260)$ are closer to P (distance: 61.8)

- nearest-neighbor queries: adjacent buckets as well for no FN
 - e.g., $P = (45, 200)$, the closest point in bucket (50,120) (distance: 80.2)
(30,260) and (60,260) are closer to P (distance: 61.8)



(45, \$200K)

Hash structures (2): partitioned hashing

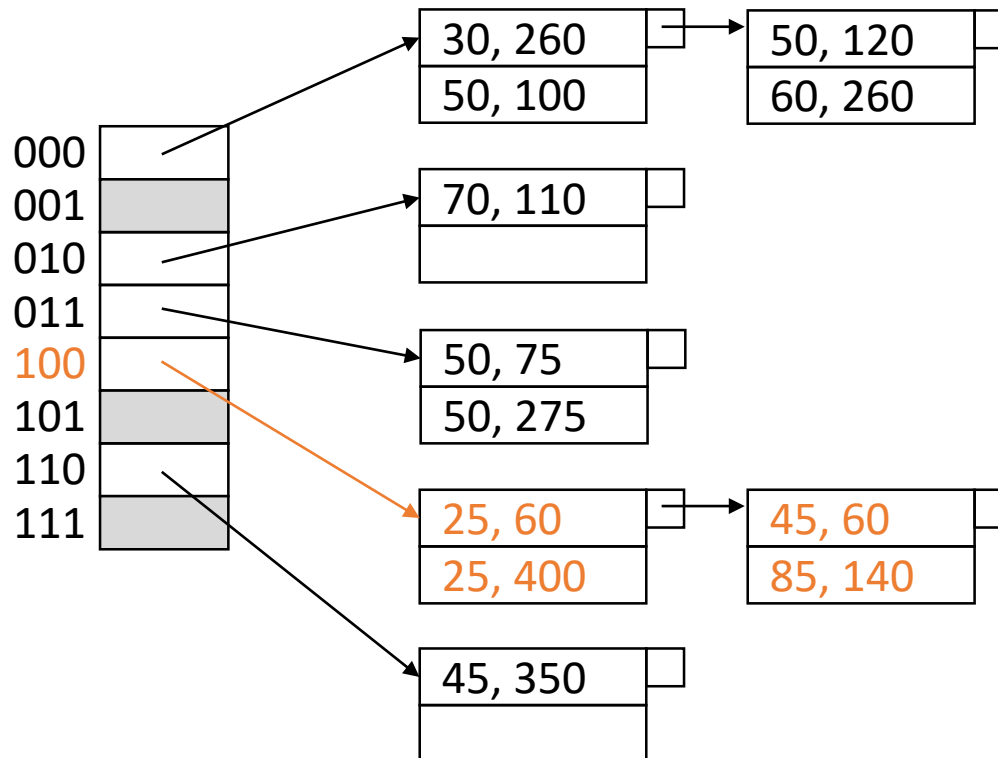
- Problem of a naïve hashing method
 - e.g., a : integer attribute, b : character-string attribute
 - we cannot compute $h(a,b)$ if either a or b is not given
- Divide k bits among n attributes
 - h : actually a list of hash functions $(h_1, h_2, \dots, h_n)$
 - h_i produces k_i bits as a hash value ($k_1 + \dots + k_n = k$)
 - bucket for a tuple with $(v_1, v_2, \dots, v_n) : h_1(v_1)h_2(v_2)\dots h_n(v_n)$
- e.g., 10-bit bucket numbers (1024 buckets)
 - four bits to attribute a , six bits to attribute b
 - given a -value A , b -value B , $h_a(A)=0101$, $h_b(B)=111000$, the tuple (A,B) hashes to bucket 0101111000
 - only given B , search 16 buckets whose number ends with $h_b(B)$

Example of partitioned hashing

- One bit to age, two bits to salary

- $h_{age}(A) = A \text{ modulo } 2$

- $h_{salary}(S) = S \text{ modulo } 4$



Comparison of grid files and partitioned hashing

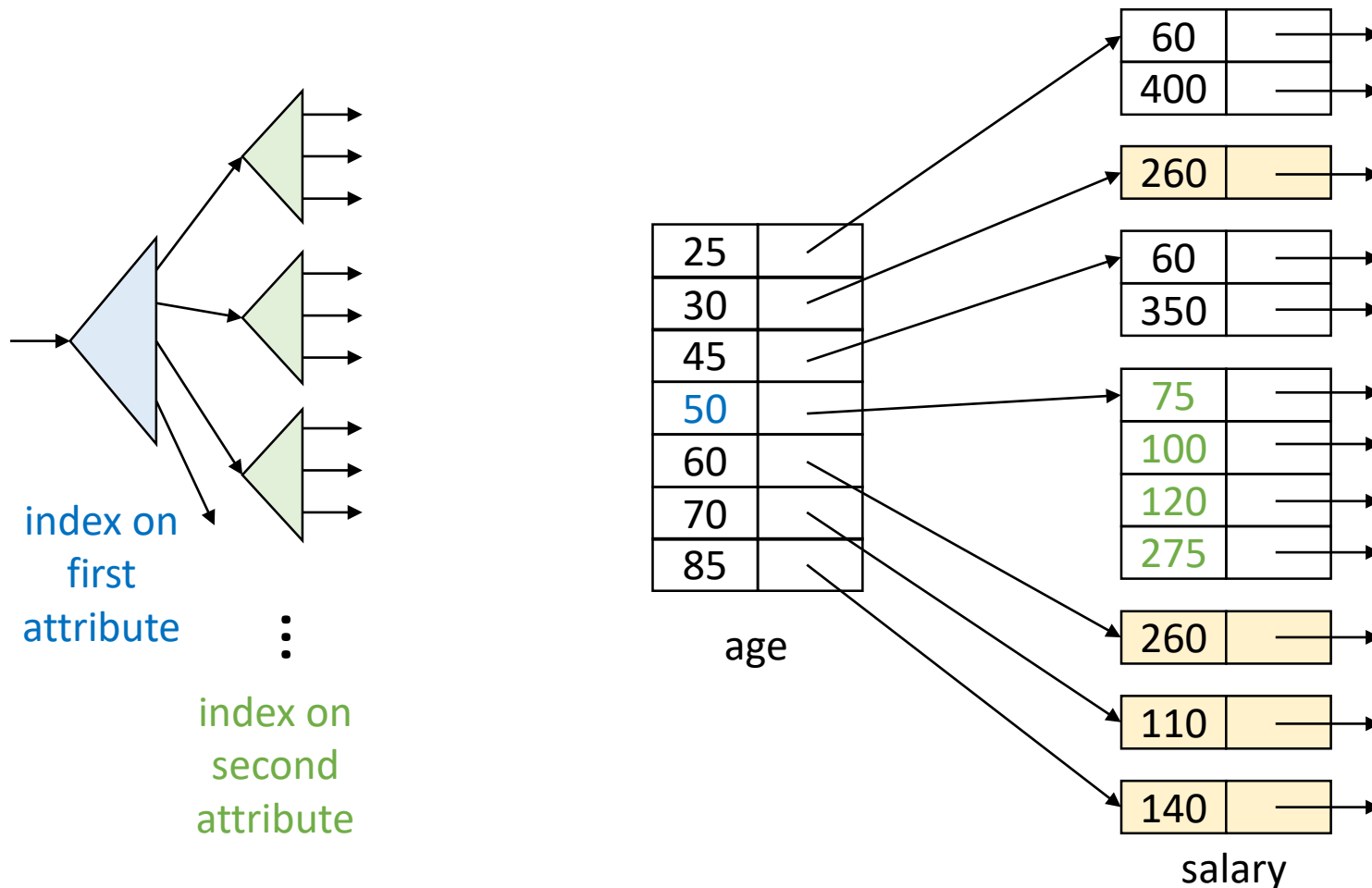
- Partitioned hashing
 - almost **useless for nearest-neighbor queries or range queries**
 - physical distance between points is not reflected by the closeness of bucket numbers
 - well chosen hash functions **make the buckets be equally occupied**
 - **better for partial match queries**
- Grid file
 - tend to **leave many buckets empty** or nearly so, especially when # of dimensions is large
 - due to some correlation among attributes (dimensions)
 - **better for nearest-neighbor queries or range queries**

Tree structures for multidimensional data

- Multiple-key indexes
 - kd-trees
 - Quad trees
 - R-trees
- for sets of **points**
- for sets of **regions** (also useful for points)

Multiple-key indexes

- Simple tree scheme: **index of indexes**
 - nodes at each level are **indexes** for one attribute
 - **index can be any type** of conventional index (B-tree, hash table)

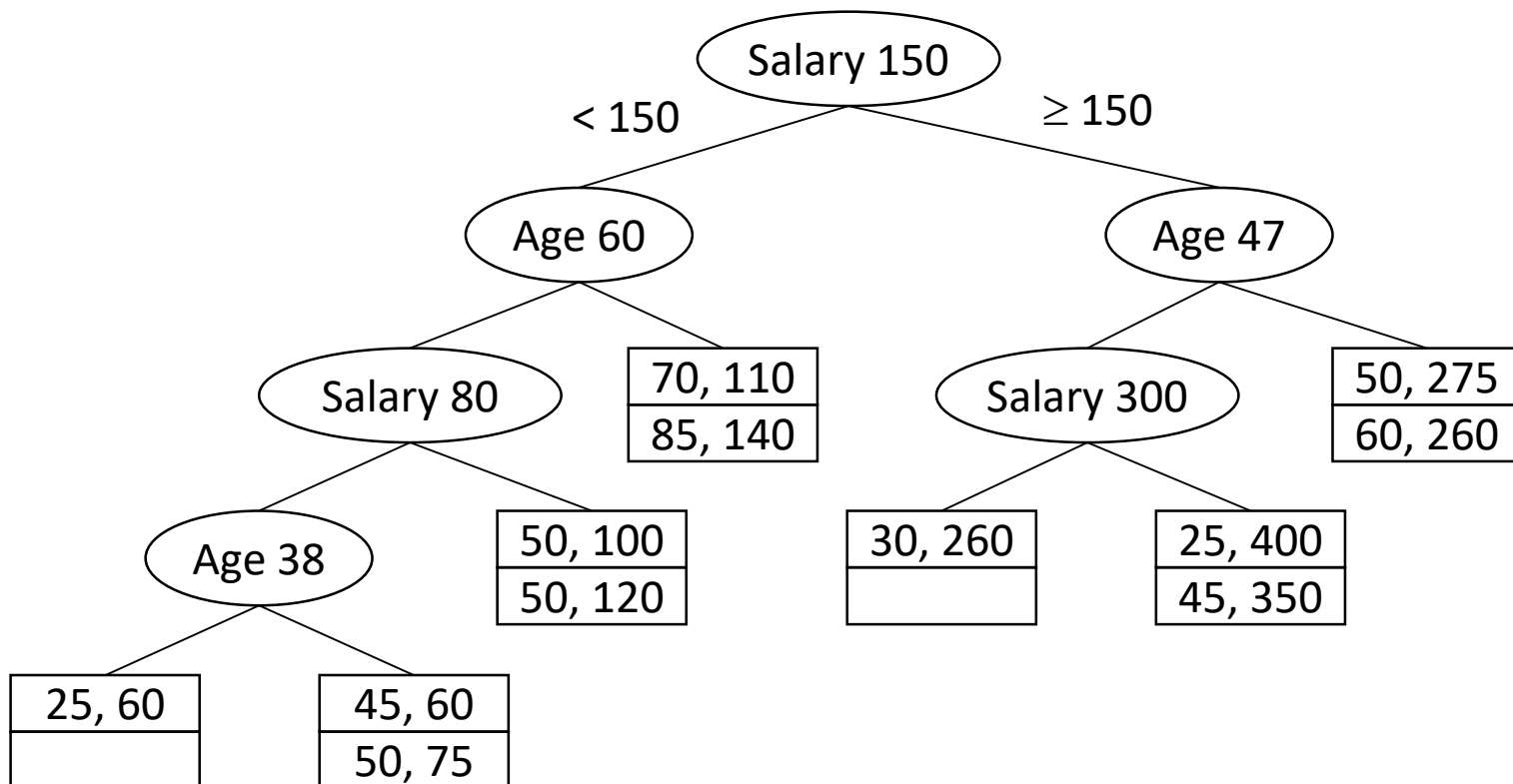


Performance of multiple-key indexes

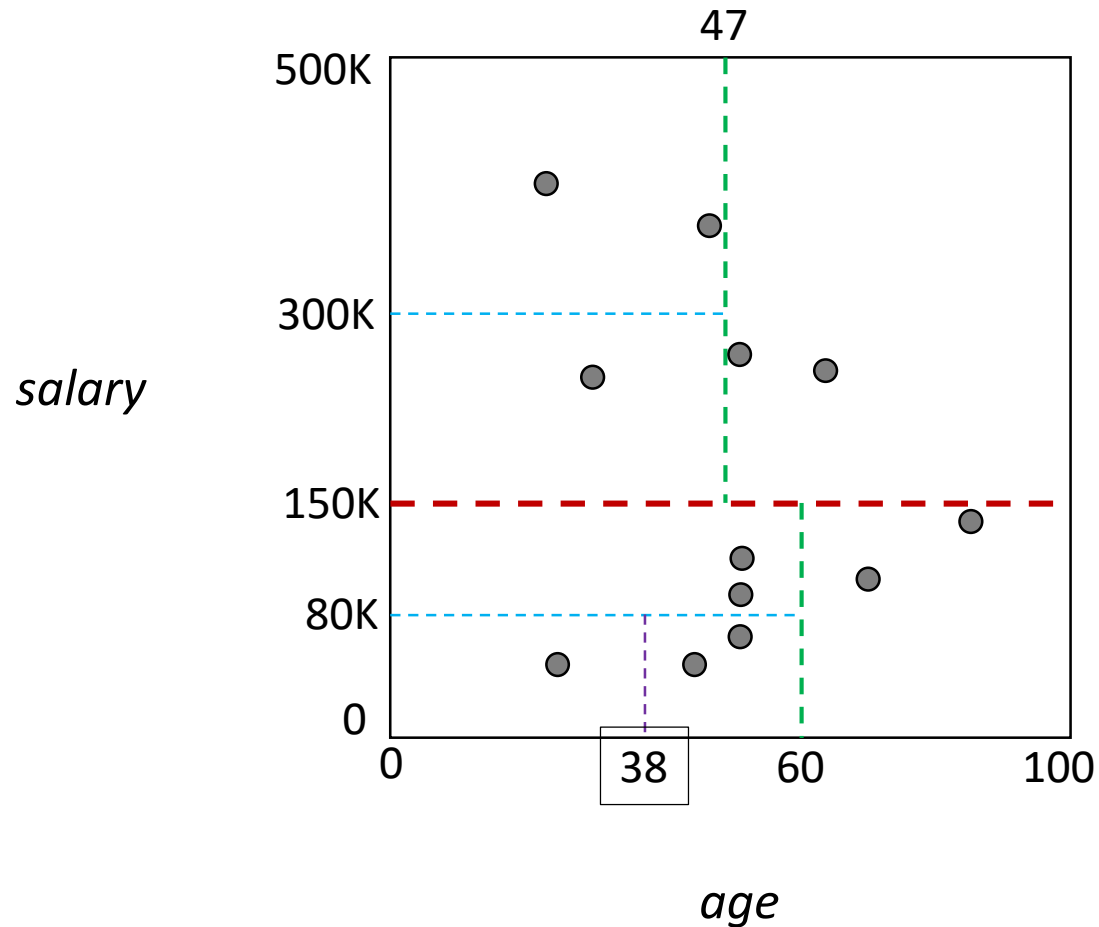
- Partial-match queries
 - access is quite efficient, if the first attribute is specified
 - must search every subindex, otherwise (time-consuming!)
- Range queries
 - efficient if individual indexes support range queries (e.g., B-tree)
- Nearest-neighbor queries
 - can be supported by a series of range queries

k-dimensional search tree (kd-trees)

- **Interior nodes**: main-memory data structure generalizing the **binary search tree** to multidimensional data
- **Leaf nodes**: data blocks



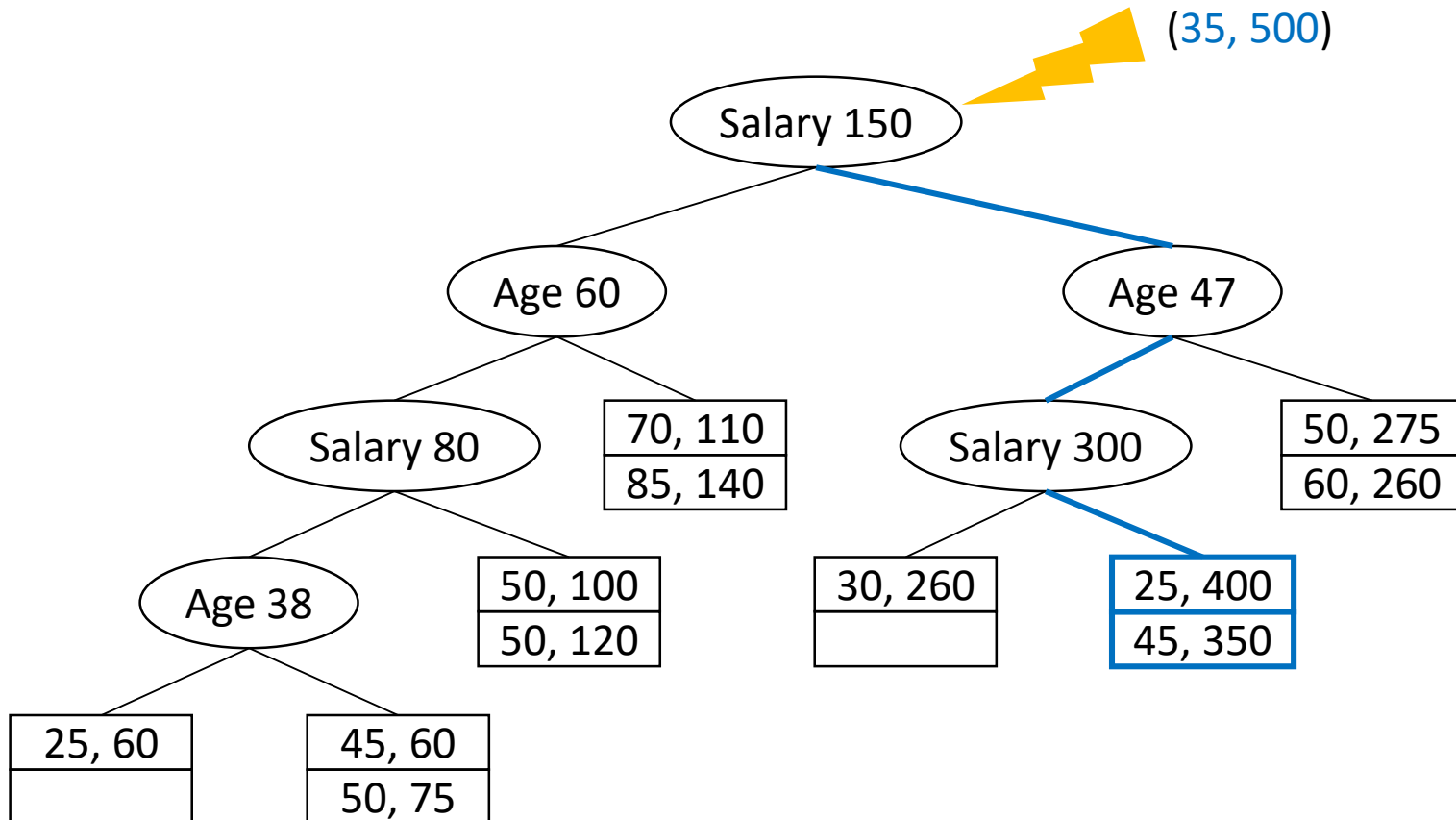
Example: partitions implied by the kd-tree



Operations on kd-trees

- Insertion

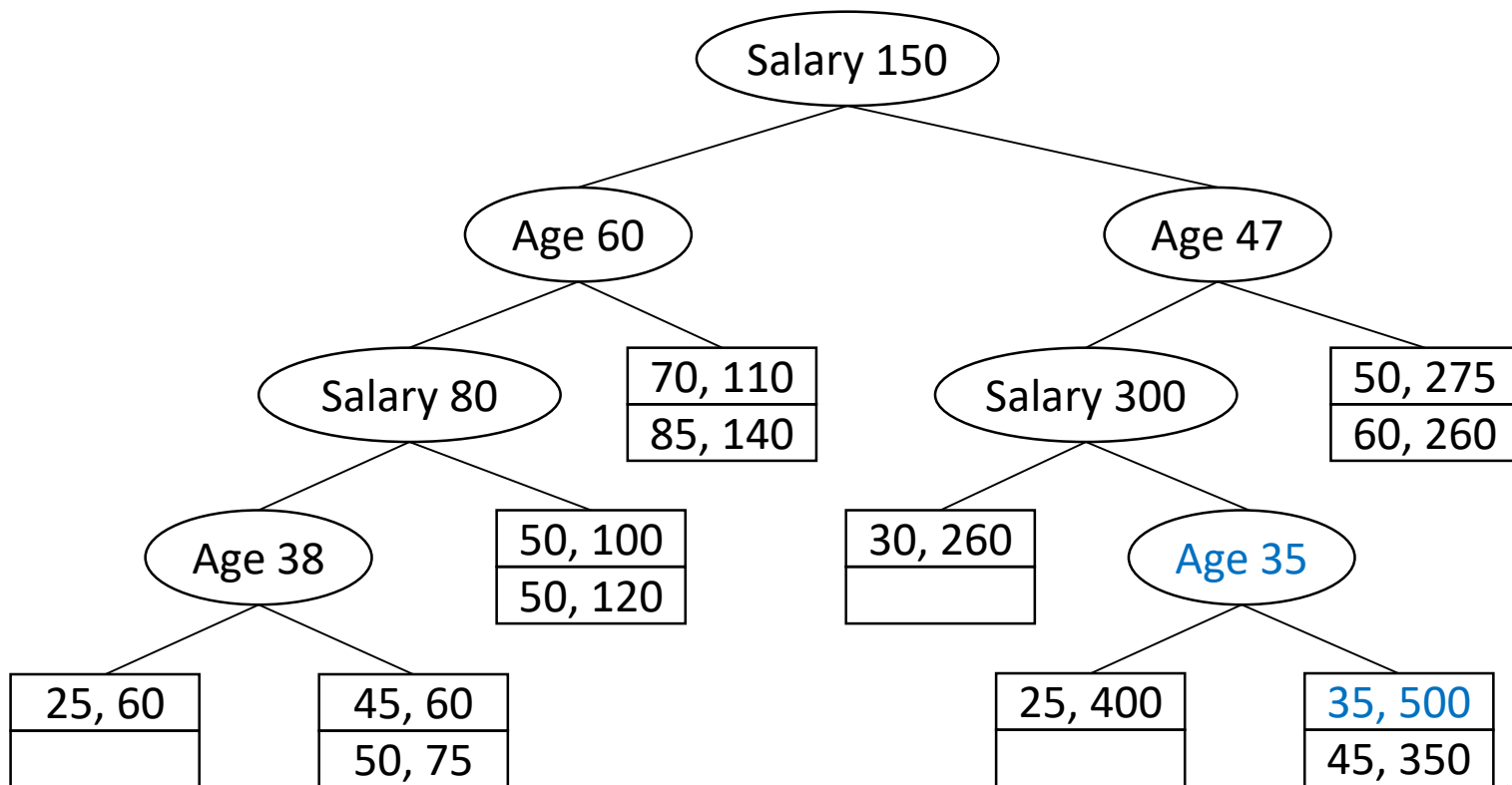
- split the leaf block and create a new interior node if there is no room



Operations on kd-trees

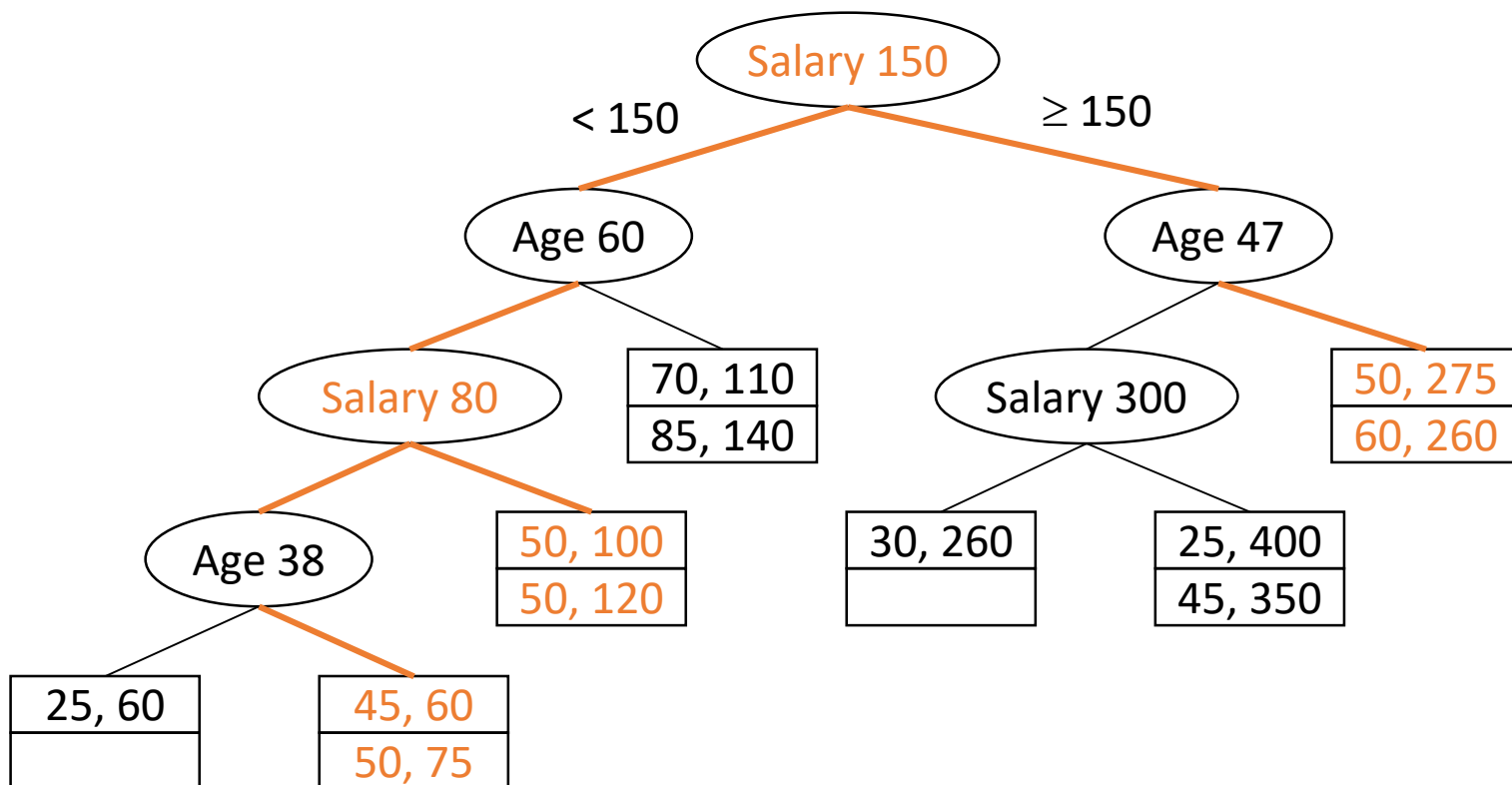
- Insertion

- split the leaf block and create a new interior node if there is no room



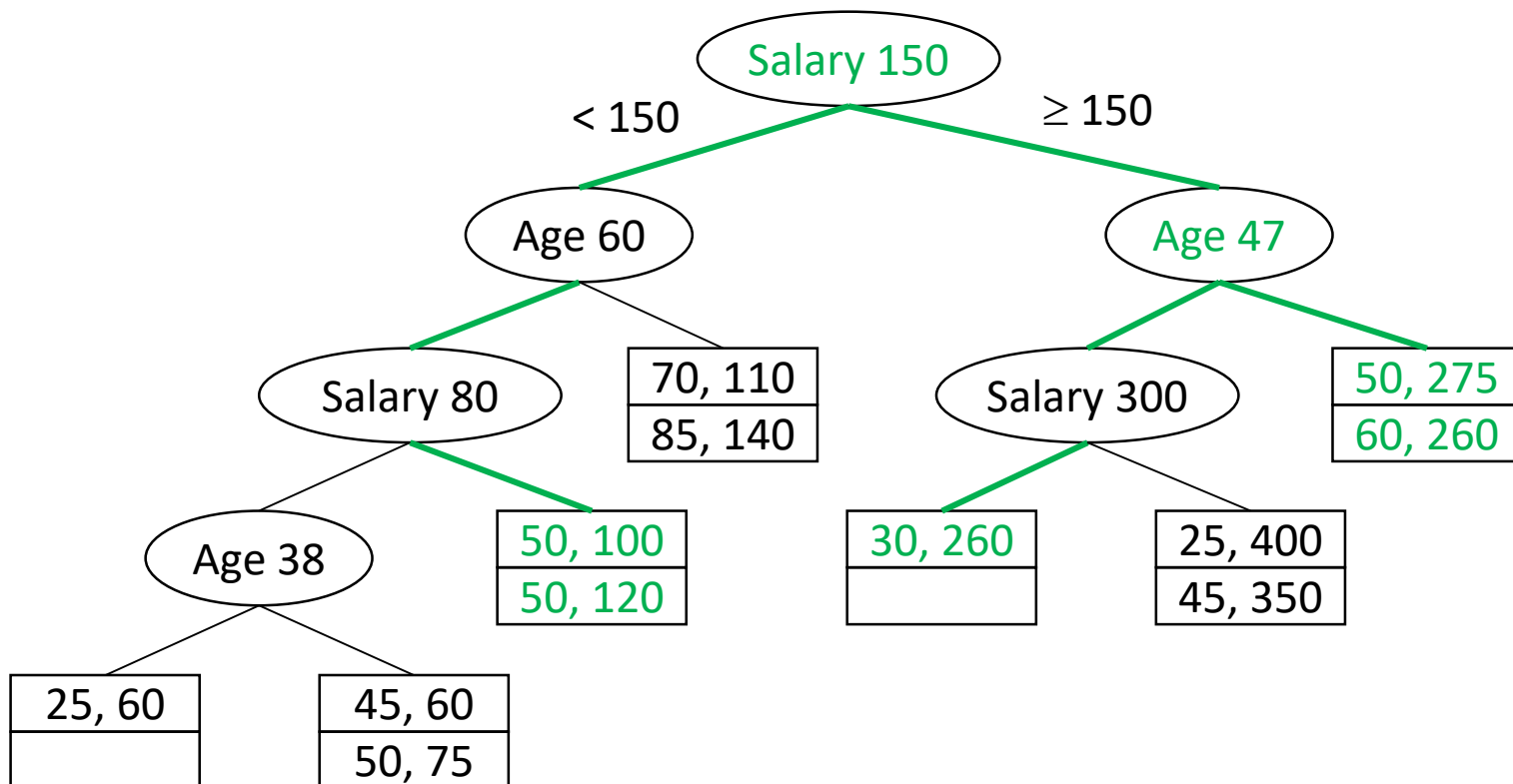
- Partial-match queries

- we **must explore both children**, if we don't know the value of the attribute at a node
- e.g., find all points with **age=50**



- Range queries

- we **must explore both children**, if the range straddles the **splitting value** at a node
- e.g., age=[35,55], salary=[100,200]



Adapting kd-trees to secondary storage

- Long-path problem

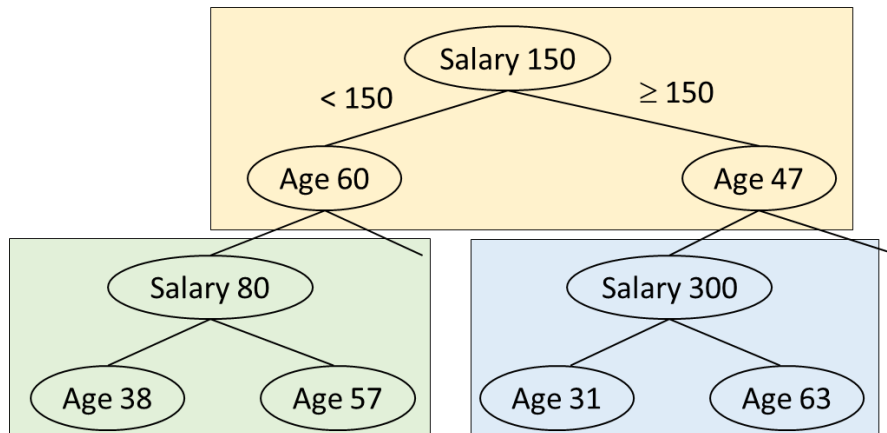
- assumption: n leaves in a kd-tree
- average length of a path from the root: $\log_2 n$
- e.g., $n=1000$, we need about 10 disk I/O's

- Multiway branches at interior nodes

- split values of an attribute into $n+1$ ranges

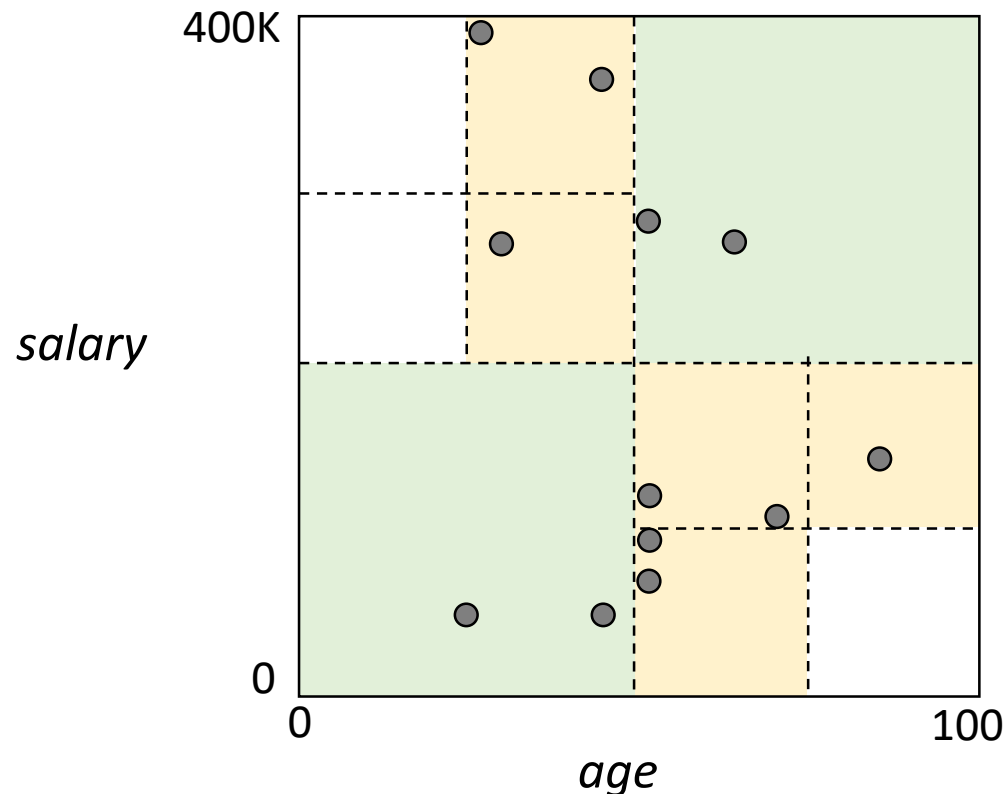
- Group interior nodes into blocks

- pack many interior nodes into a single block

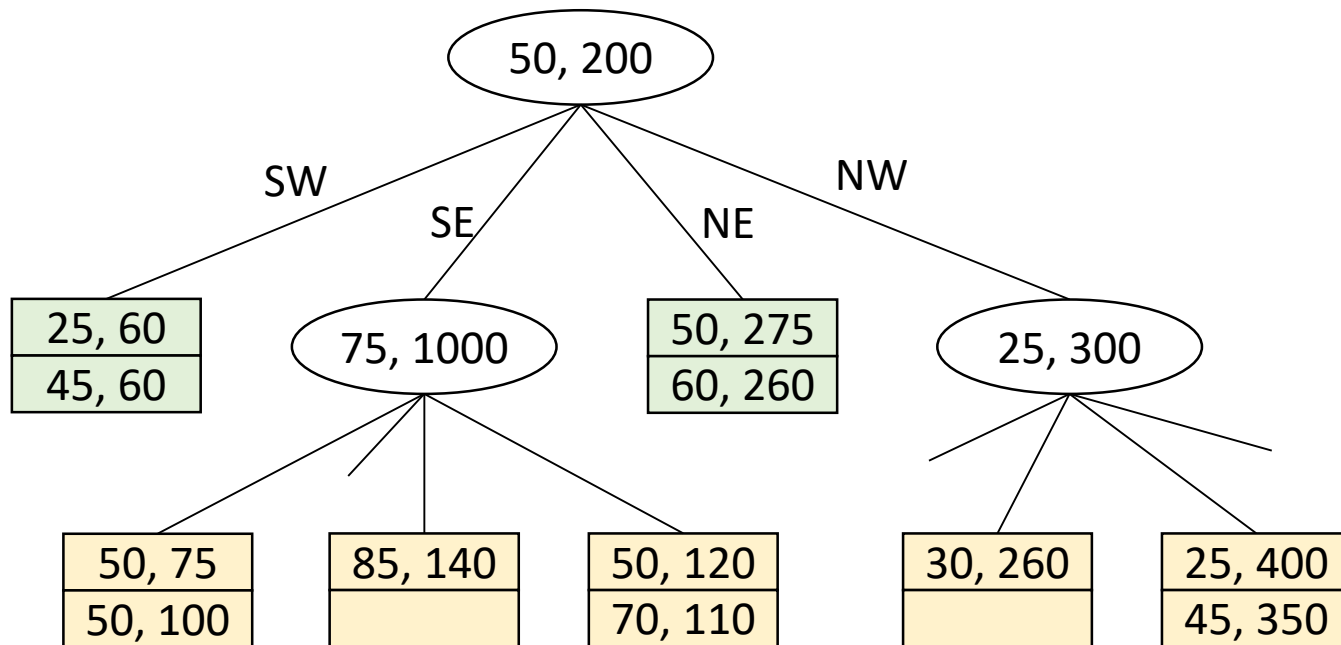


Quad trees

- Each interior node corresponds to a square region in two dimensions (or k -dimensional cube in k dimensions)
- e.g., assumption: two records per block

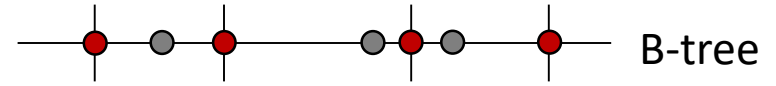


- Interior nodes in k dimensions have 2^k children
 - e.g., $k=7$ is a convenient # of dimensions, if $2^7=128$ pointers can fit in a block
 - but, many null pointers (empty quadrants) in interior nodes when # of dimensions is large
 - k is also given in many cases
 - situation when $k=2$ is similar to kd-trees

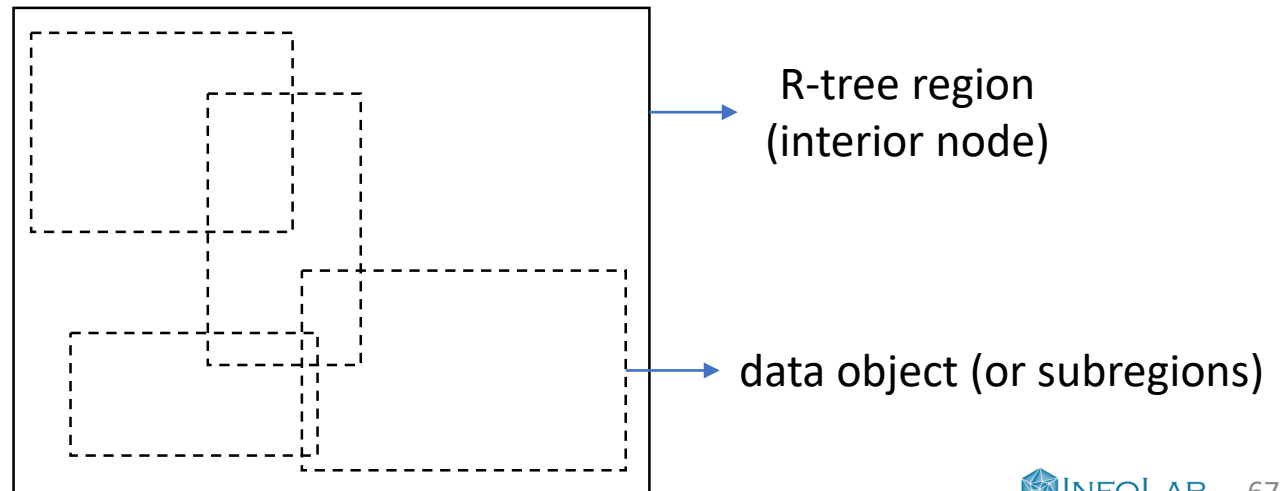


* SW: southwest quadrant

Region tree (**R-trees**)



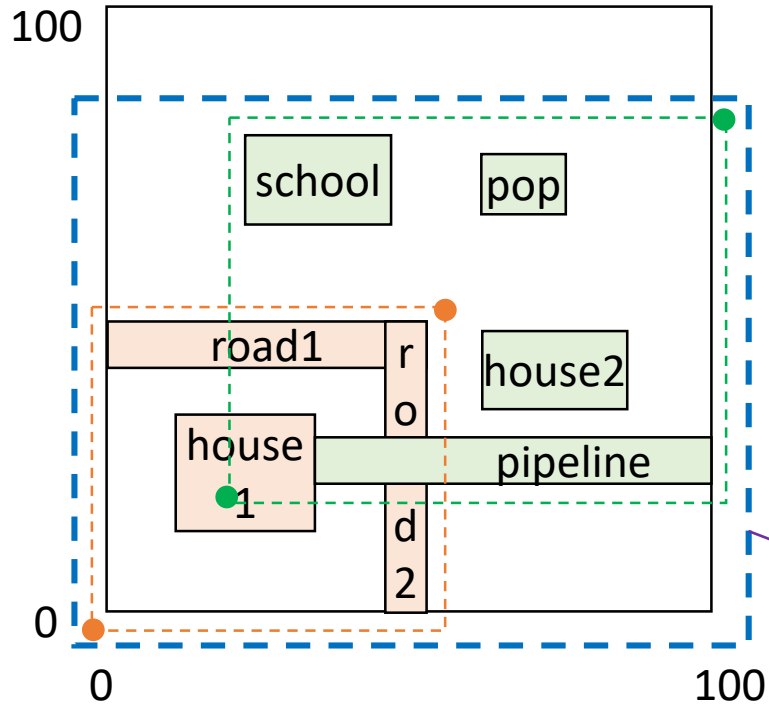
- Capture the **spirit of B-tree** for multidimensional data
- Interior node corresponds to some **(interior) region**
 - subregions represent the contents of its children
 - subregions **can overlap** (desirable to keep the overlap small)
 - subregions **usually do not cover the entire region**



Operations on R-trees

- **Where-am-I query** (typical query using R-tree)
 - query: find data regions where a given point P lies
 - ① start at the root (representing the entire region)
 - ② determine which children of the root correspond to interior regions that contain the point P
 - zero, one, or several such regions
 - ③ if there is at least one region containing P , recursively search for P at the regions
 - ④ find the actual data regions (records, or pointers) at leaves

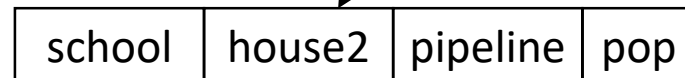
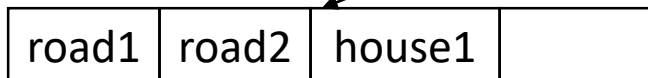
Example



MBR (Minimum Bounding Rectangle) :

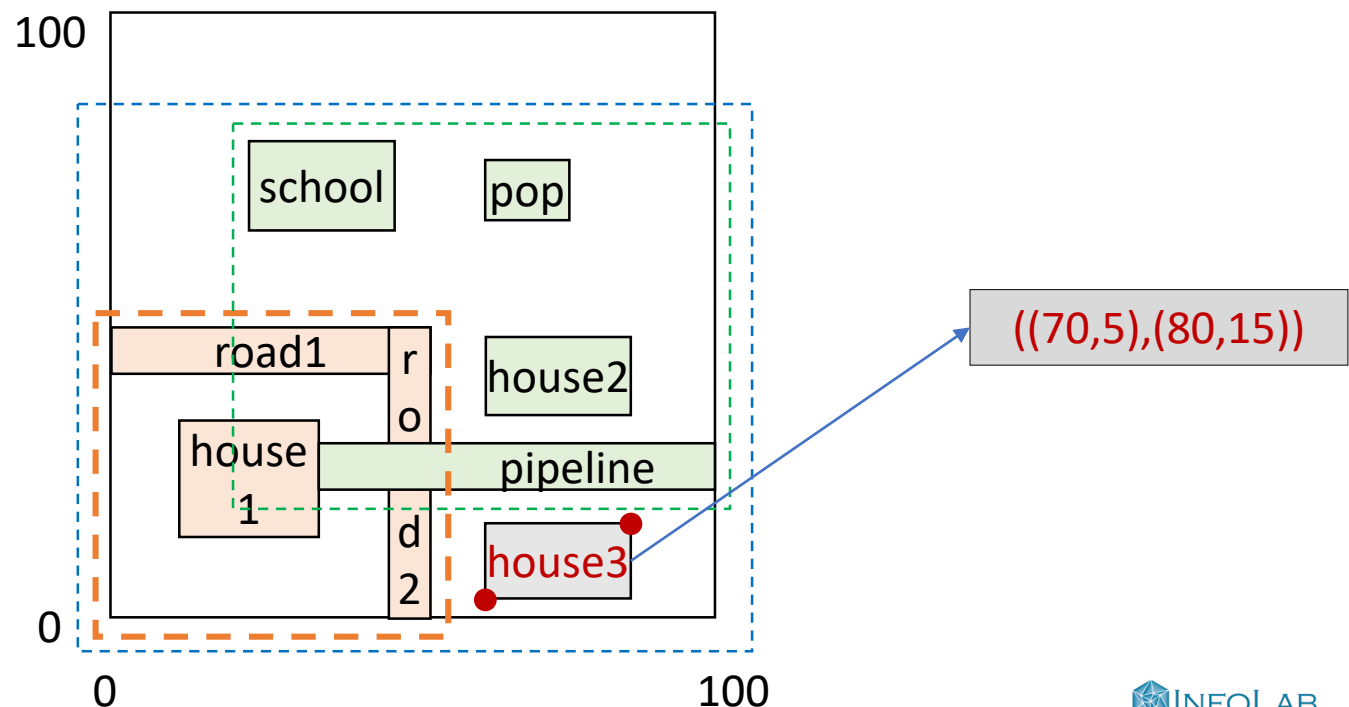
encloses the data objects within minimally

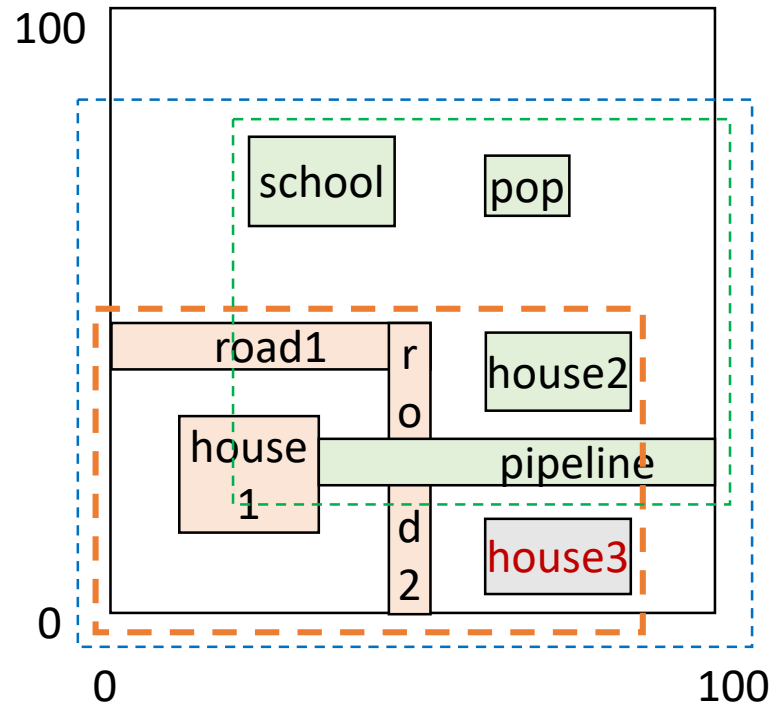
$P=(25,25)$



four data regions per leaf

- **Insertion** (a new region R): **find subregion(s)** where R fits
 - if there is more than one region, **pick one and go to the child**
 - if there is no subregion containing R, **expand one of the subregions**
 - **which one to pick may be a difficult decision**
 - try to expand regions as little as possible
 - increase the area of overlap between subregions as little as possible
 - if there is no room at a leaf, split the leaf





$((70,5),(80,15))$

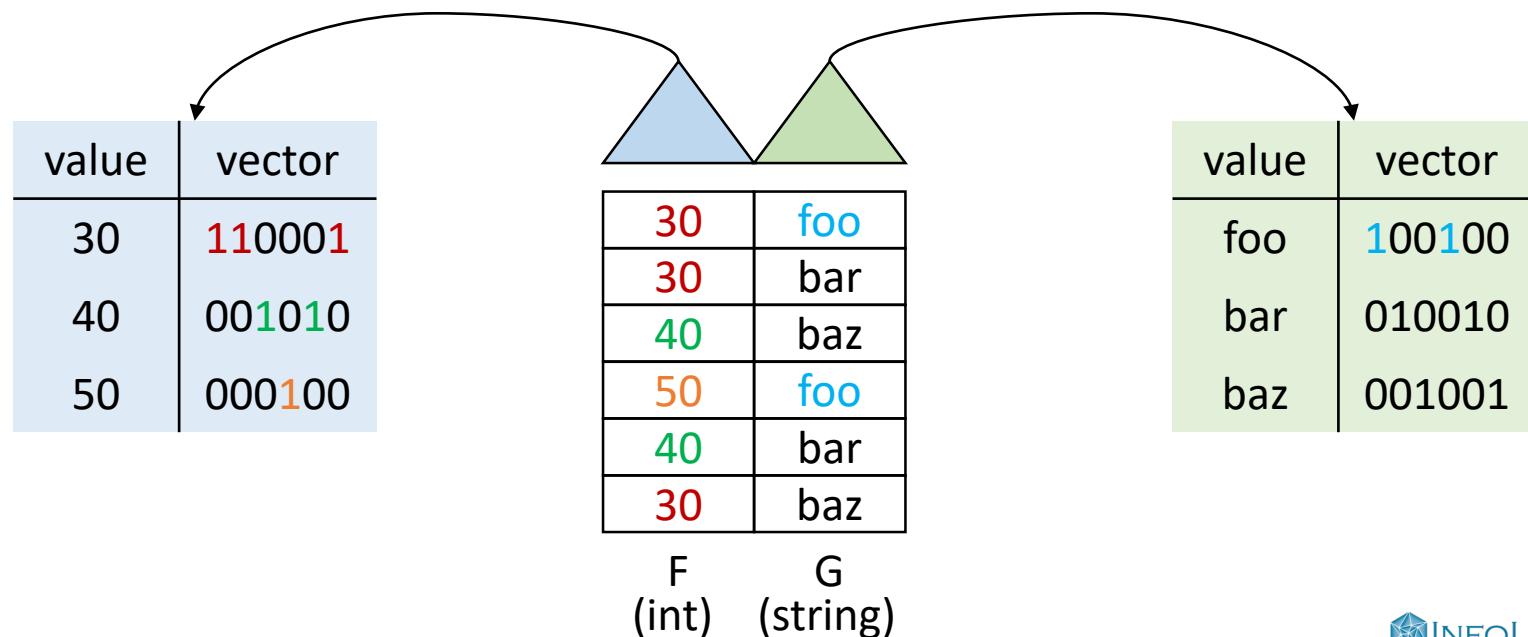


road1	road2	house1	house3
-------	-------	--------	--------

school	house2	pipeline	pop
--------	--------	----------	-----

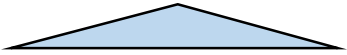
Bitmap indexes

- Bitmap index for a field F: collection of bit-vectors of length n
 - n : # of records
 - m : # of bit-vectors
 - vector for value v has 1 in position i , if the i -th record has v in field F
 - vector has 0 there, otherwise

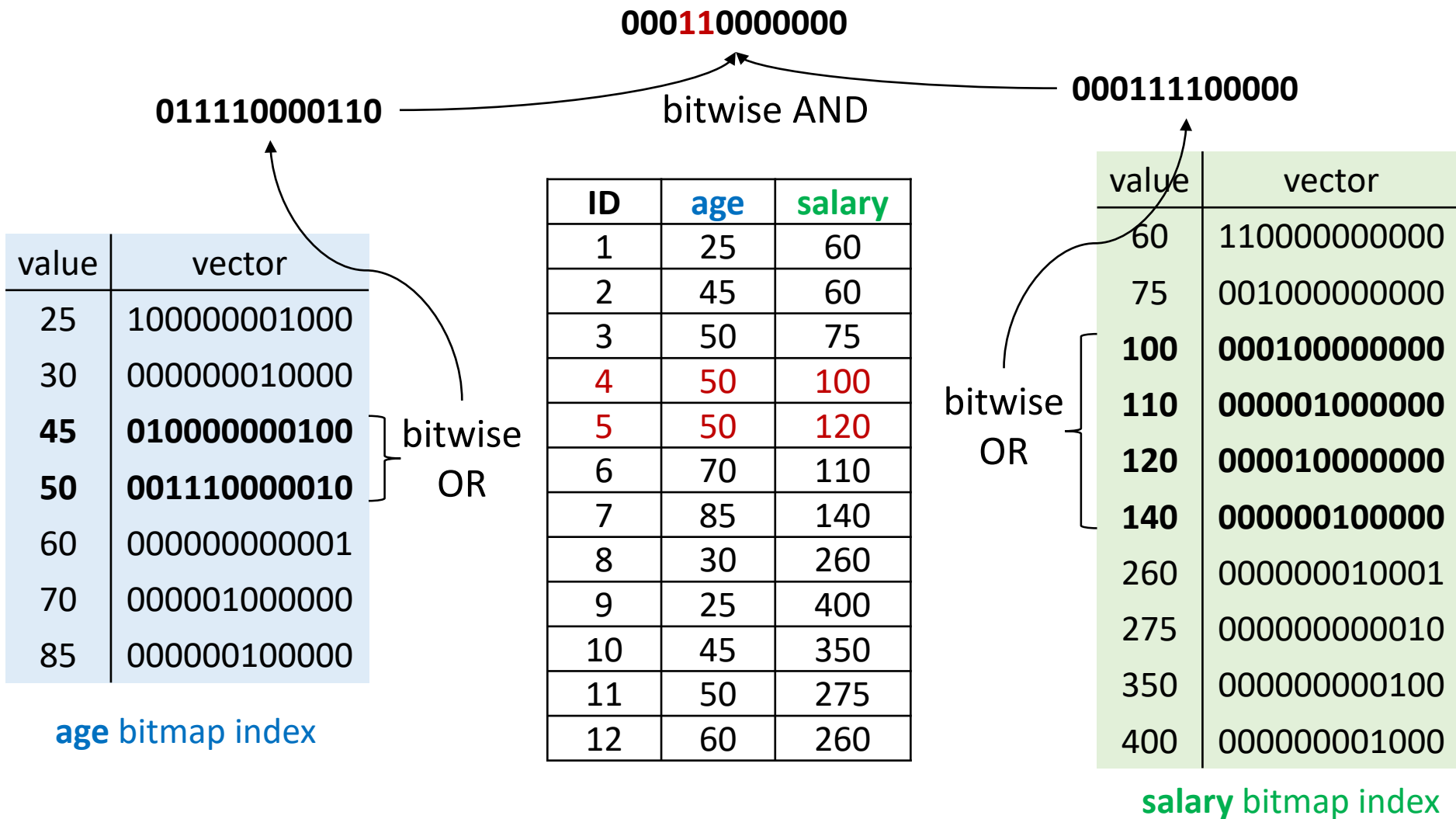


Motivation for bitmap indexes

- Space complexity
 - n^2 bits, if the field is a key (i.e., $m=n$), and there are n records
 - compression can be used to make # of bits closer to n
- Answer partial-match queries very efficiently
 - e.g., bitmap indexes on both studioName and year intersecting two bit vectors using bitwise AND

```
SELECT title
FROM Movie
WHERE  studioName='Disney' AND  year=2005;
```

- Answer **range queries**
 - e.g., **age**=[45,55], **salary**=[100,200]



Compressed bitmaps (1)

- A bitmap index on field F
- Assumption: m different values, n records
 - # of bits : mn
 - block: 4,096 bytes (i.e., 32,768 bits per block)
 - # of blocks for bitmap index = $mn / 32768$
- Compared to a data file, the size of a bitmap index is
 - case 1: small, if m is small
 - case 2: large, if m is large (we solve this case by compression)

Compressed bitmaps (2) : encoding

- Encoding bitmap

- if m is large, 1's in a bit-vector will be very rare
- probability that any bit is 1 : $1 / m$
- opportunity to encode bit-vectors (run-length encoding)

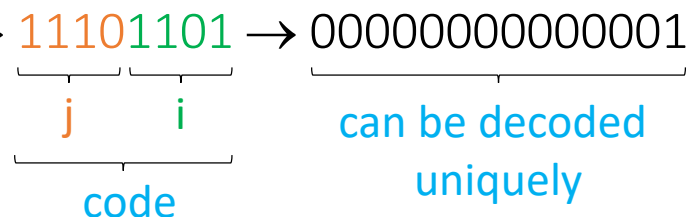
- **run**: a sequence of i 0's followed by a 1

- naive method: e.g., 000101 $\rightarrow$ 111, 010001 $\rightarrow$ 111
- problem: cannot be decoded uniquely into one bit-vector

- j : # of bits required for the binary representation of i

concatenated {

- the number j is represented in “unary” by $j-1$ 1's and a single 0
- the number i is represented in binary

- e.g., if $i = 13$, $j = \log_2 i = 4$, and so, $13 \rightarrow$ 11101101 $\rightarrow$ 000000000000001
 - e.g., if $i = 1$, $j = 1$, and so, $1 \rightarrow$ 01
 - e.g., if $i = 0$, $j = 1$, and so, $0 \rightarrow$ 00
- 

Compressed bitmaps (3) : decoding

- Decoding bitmap

- ① scan forward to the first 0 to determine the value of j
- ② look at the next j bits (i is the integer there in binary)
- ③ repeat Steps 1 and 2

– e.g., 1 1 1 0 1 1 0 1 0 0 1 0 1 1

- 1st run: $j = 4$ from 1110, $i = 1101 = 13$
- 2nd run: $j = 1$ from 0, $i = 0$
- 3rd run: $j = 2$ from 10, $i = 11 = 3$

- reconstruct the original bit-vector: 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 1

Compressed bitmaps (4) : space complexity

- Size of bitmap index
 - if $m=n$, each value occurs once (e.g., F is a key)
 - a single run (length of each run cannot be longer than n)
 - longest run: 0000 0001 (length n)
 - maximum length of a code for a run of length n : $2\log_2 n$ bits
 - e.g., if $i = 13$, $j = \log_2 i = 4$, and so, $13 \rightarrow 11101101$
 - size of bitmap index: $2n\log_2 n$ (upper bound) $\ll n^2$ (i.e., nm)

Operating on run-length-encoded bit-vectors

value	vector	compressed vector
25	1 0000000 1 000	00 110111
30	0000000 1 0000	110111

n = 12

00**110111** → **0**, **7**

110111 → **7**

OR

0 1 2 3 4 5 6 7 8 9
→ **1****0000000****1****1****000**

Finding bit-vectors

- Issue

- when we want to find the bit-vector for a given value, or the bit-vectors corresponding to values in a given range, **how do we find them efficiently?**

- Method

- think of each bit-vector as **a record of (key, bit-vector)**
 - key : the value corresponding to the bit-vector
- store bit-vectors as **variable-length records**
 - they will grow as more records are added to the data file
- build **a secondary index** to find the bit-vectors given a value(s)



key	bit-vector
25	00110111
30	110111

Finding data records

- Issue

- when we have selected a set of records that answer our query, how do we retrieve those records efficiently?

- Method

- add a column for the **record ID** in the data file
- build a **secondary index** to find the **k-th record** in the data file



key	bit-vector
25	00110111
30	110111

bitmap index

$k = \{1, 9\}$
for key=25

100000001000
000000010000



original columns

ID	value	other columns
1	25	
2		
3		
4		
5		
6		
7		
8	30	
9	25	
10		
11		
12		

data file

Handling modifications to the data file (1)

- Issue
 - when the data file changes by **insertion** or **deletion** of records, how do we adjust the bitmap index on a given field?
- Two aspects to be considered
 - ① record IDs must remain fixed once assigned
 - ② changes to the data file require the bitmap index to change as well

Handling modifications to the data file (2)

- Deletion of record i

- set tombstone in the space of record i
- find the appropriate bit-vector using the field value of record i
- change the 1 at position i in the bit-vector to 0



key	bit-vector
25	00
30	110111

10000000000000
0000000010000

bitmap index

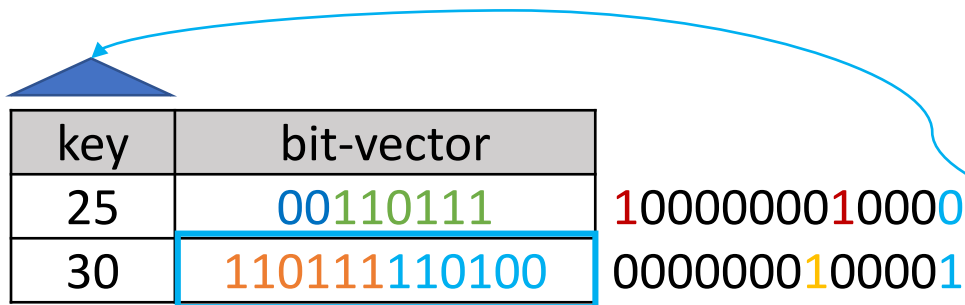
data file

key	value	other columns
1	25	
2		
3		
4		
5		
6		
7		
8	30	
tombstone	25	
10		
11		
12		

Handling modifications to the data file (3)

- Insertion of record *i*

- keep track of the next available record number and assign it to the new record
- modify the bit-vector for the corresponding value of the new record by appending a 1 at the end



key	bit-vector
25	00110111
30	110111110100

1000000010000
0000000100001

bitmap index

data file

key	value	other columns
1	25	
2		
3		
4		
5		
6		
7		
8	30	
9	25	
10		
11		
12		
13	30	

Thank you

Q & A